

```

1  # === File: app/main.py ===
2  """FastAPI application entry point with production middleware stack."""
3  from __future__ import annotations
4  from contextlib import asynccontextmanager
5  from typing import Any
6  from fastapi import FastAPI, Request
7  from fastapi.middleware.cors import CORSMiddleware
8  from fastapi.middleware.trustedhost import TrustedHostMiddleware
9  from fastapi.responses import JSONResponse
10 from app.api.v1.router import router as v1_router
11 from app.config import settings
12 from app.utils.exceptions import AgentFlowError, error_response
13 from app.utils.logger import setup_logging
14 from app.core.middleware import (
15     APIKeyAuthMiddleware,
16     RequestIDMiddleware,
17     SecurityHeadersMiddleware,
18 )
19 from app.core.observability.metrics import MetricsMiddleware, get_metrics_response
20 from app.core.settings_guard import enforce_production_settings
21 setup_logging()
22 try:
23     from slowapi import Limiter, _rate_limit_exceeded_handler
24     from slowapi.util import get_remote_address
25     HAS_SLOWAPI = True
26 except ImportError:
27     HAS_SLOWAPI = False
28 def _sqlite_table_columns(connection, table: str) -> set[str]:
29     from sqlalchemy import text
30     rows = connection.execute(text(f"PRAGMA table_info({table})")).fetchall()
31     return {row[1] for row in rows}
32 def _ensure_sqlite_columns(connection) -> None:
33     """Add missing columns on existing SQLite DBs (create_all never ALTERs)."""
34     from sqlalchemy import text
35     import logging
36     log = logging.getLogger(__name__)
37     if connection.dialect.name != "sqlite":
38         return
39     task_cols = {
40         "celery_task_id": "ALTER TABLE tasks ADD COLUMN celery_task_id VARCHAR(255)",
41         "is_archived": "ALTER TABLE tasks ADD COLUMN is_archived BOOLEAN NOT NULL DEFAULT",
42         "created_by": "ALTER TABLE tasks ADD COLUMN created_by VARCHAR(100) NOT NULL DEFAU",
43         "tenant_id": "ALTER TABLE tasks ADD COLUMN tenant_id VARCHAR(36)",
44     }
45     existing = _sqlite_table_columns(connection, "tasks")
46     for name, ddl in task_cols.items():
47         if name not in existing:
48             connection.execute(text(ddl))
49             log.info("SQLite backfill: tasks.%s", name)
50     if "audit_logs" in {
51         r[0]
52         for r in connection.execute(
53             text("SELECT name FROM sqlite_master WHERE type='table'")
54         ).fetchall()
55     }:

```

```

73         al_cols = {
74             "tenant_id": "ALTER TABLE audit_logs ADD COLUMN tenant_id VARCHAR(36)",
75         }
76         existing = _sqlite_table_columns(connection, "audit_logs")
77         for name, ddl in al_cols.items():
78             if name not in existing:
79                 connection.execute(text(ddl))
80                 log.info("SQLite backfill: audit_logs.%s", name)
81     if "traces" in {
82         r[0]
83         for r in connection.execute(
84             text("SELECT name FROM sqlite_master WHERE type='table'")
85         ).fetchall()
86     }:
87         trace_cols = {
88             "prompt_tokens": "ALTER TABLE traces ADD COLUMN prompt_tokens INTEGER NOT NULL",
89             "completion_tokens": "ALTER TABLE traces ADD COLUMN completion_tokens INTEGER",
90             "cost": "ALTER TABLE traces ADD COLUMN cost FLOAT NOT NULL DEFAULT 0",
91             "agent_version": "ALTER TABLE traces ADD COLUMN agent_version VARCHAR(100)",
92             "prompt_version": "ALTER TABLE traces ADD COLUMN prompt_version VARCHAR(100)",
93             "model_version": "ALTER TABLE traces ADD COLUMN model_version VARCHAR(100)",
94             "tool_version": "ALTER TABLE traces ADD COLUMN tool_version VARCHAR(100)",
95         }
96         existing = _sqlite_table_columns(connection, "traces")
97         for name, ddl in trace_cols.items():
98             if name not in existing:
99                 connection.execute(text(ddl))
100                 log.info("SQLite backfill: traces.%s", name)
101     if "metric_scores" in {
102         r[0]
103         for r in connection.execute(
104             text("SELECT name FROM sqlite_master WHERE type='table'")
105         ).fetchall()
106     }:
107         ms_cols = {
108             "confidence": "ALTER TABLE metric_scores ADD COLUMN confidence FLOAT",
109             "is_human_reviewed": "ALTER TABLE metric_scores ADD COLUMN is_human_reviewed B",
110             "human_score": "ALTER TABLE metric_scores ADD COLUMN human_score FLOAT",
111             "reviewer": "ALTER TABLE metric_scores ADD COLUMN reviewer VARCHAR(100)",
112         }
113         existing = _sqlite_table_columns(connection, "metric_scores")
114         for name, ddl in ms_cols.items():
115             if name not in existing:
116                 connection.execute(text(ddl))
117                 log.info("SQLite backfill: metric_scores.%s", name)
118 @asynccontextmanager
119 async def lifespan(app: FastAPI):
120     """Manage application lifecycle."""
121     from app.models.base import Base
122     from app.core.dependencies import engine
123     import logging
124     log = logging.getLogger(__name__)
125     try:
126         enforce_production_settings(settings)
127     except Exception:

```

```

145         log.exception("Production settings validation failed")
146         raise
147     import app.models
148     async with engine.begin() as conn:
149         await conn.run_sync(Base.metadata.create_all)
150         try:
151             await conn.run_sync(_ensure_sqlite_columns)
152         except Exception as exc:
153             log.warning("Schema backfill skipped: %s", exc)
154     log.info("Tables initialized")
155     ws_started = False
156     try:
157         from app.core.ws_hub import start_ws_hub, stop_ws_hub
158         await start_ws_hub()
159         ws_started = True
160     except Exception as exc:
161         log.warning("WS hub start failed: %s", exc)
162     if getattr(settings, "CACHE_WARMUP_ON_STARTUP", False):
163         try:
164             from app.core.cache.warmup import warm_cache
165             summary = await warm_cache(actor="anonymous", limit=20)
166             log.info("Cache warmup: %s", summary)
167         except Exception as exc:
168             log.warning("Cache warmup skipped: %s", exc)
169     try:
170         from app.core.profiles import apply_profile_from_settings, profile_status
171         apply_profile_from_settings()
172         log.info("Infrastructure ports: %s", profile_status())
173     except Exception as exc:
174         log.warning("Deploy profile apply failed (using lazy defaults): %s", exc)
175     plugins_bootstrapped = False
176     try:
177         from app.core.plugins.manager import get_plugin_manager
178         from app.core.plugins.market import get_plugin_market
179         if getattr(settings, "PLUGINS_ENABLED", True):
180             mgr = get_plugin_manager()
181             strict = bool(
182                 getattr(settings, "PLUGIN_STRICT_ALLOWLIST", False)
183                 or getattr(settings, "PLUGIN_STRICT_MODE", False)
184             )
185             modules = list(getattr(settings, "plugin_module_list", []) or [])
186             directories = list(getattr(settings, "plugin_dir_list", []) or [])
187             if strict:
188                 directories = []
189                 if not modules:
190                     log.warning(
191                         "PLUGIN_STRICT_MODE/ALLOWLIST=true but PLUGIN_MODULES empty – no p
192                     )
193             try:
194                 from app.core.plugins.signature import (
195                     filter_signed_modules,
196                     signature_check_enabled,
197                 )
198                 if signature_check_enabled() and modules:
199                     modules, rejected = filter_signed_modules(modules)

```

```

217         if rejected:
218             log.warning("Plugin signature rejections: %s", rejected)
219     except Exception as sig_exc:
220         log.warning("Plugin signature check skipped: %s", sig_exc)
221     summary = mgr.bootstrap(
222         enabled=True,
223         directories=directories,
224         modules=modules,
225         auto_activate=True,
226         allowlist=list(getattr(settings, "plugin_allowlist", []) or [])
227         or (modules if strict else None),
228     )
229     plugins_bootstrapped = True
230     log.info(
231         "Plugins bootstrap (strict=%s): %s",
232         strict,
233         summary,
234     )
235     market = get_plugin_market()
236     if getattr(settings, "PLUGIN_MARKET_SEED_EXAMPLES", True):
237         market.seed_built_in_catalog()
238     catalog = getattr(settings, "PLUGIN_CATALOG_PATH", "") or ""
239     if catalog:
240         market.catalog_path = __import__("pathlib").Path(catalog)
241         market.load_catalog()
242     else:
243         log.info("Plugin system disabled (PLUGINS_ENABLED=false)")
244     except Exception as exc:
245         log.warning("Plugin bootstrap skipped: %s", exc)
246     yield
247     if plugins_bootstrapped:
248         try:
249             from app.core.plugins.manager import get_plugin_manager
250             get_plugin_manager().shutdown()
251         except Exception as exc:
252             log.warning("Plugin shutdown failed: %s", exc)
253     if ws_started:
254         try:
255             from app.core.ws_hub import stop_ws_hub
256             await stop_ws_hub()
257         except Exception as exc:
258             log.warning("WS hub stop failed: %s", exc)
259     await engine.dispose()
260     from app.core.dependencies import close_redis
261     await close_redis()
262     app = FastAPI(
263         title=settings.APP_NAME,
264         description="Agent automated evaluation platform",
265         version="1.0.0",
266         lifespan=lifespan,
267         docs_url="/docs" if not settings.is_prod else None,
268         redoc_url="/redoc" if not settings.is_prod else None,
269     )
270     if HAS_SLOWAPI and settings.RATE_LIMIT_ENABLED:
271         limiter = Limiter(

```

```

289         key_func=get_remote_address, default_limits=[settings.RATE_LIMIT_DEFAULT]
290     )
291     app.state.limiter = limiter
292     app.add_exception_handler(429, _rate_limit_exceeded_handler)
293     app.add_middleware(APIKeyAuthMiddleware)
294     app.add_middleware(MetricsMiddleware)
295     app.add_middleware(SecurityHeadersMiddleware)
296     app.add_middleware(RequestIDMiddleware)
297     app.add_middleware(
298         CORSMiddleware,
299         allow_origins=settings.CORS_ORIGINS,
300         allow_credentials=True,
301         allow_methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"],
302         allow_headers=[
303             "Authorization",
304             "Content-Type",
305             "X-Request-ID",
306             "X-Trace-ID",
307             "X-API-Key",
308         ],
309         expose_headers=["X-Request-ID", "X-Trace-ID", "X-Error-ID"],
310     )
311     if settings.is_prod:
312         app.add_middleware(TrustedHostMiddleware, allowed_hosts=["*"])
313     app.include_router(v1_router)
314     @app.exception_handler(AgentFlowError)
315     async def agentflow_error_handler(
316         request: Request, exc: AgentFlowError
317     ) -> JSONResponse:
318         rid = getattr(request.state, "request_id", None)
319         error_id = getattr(request.state, "error_id", None)
320         if not error_id and exc.status_code >= 500:
321             try:
322                 from app.core.observability.aols import new_error_id
323                 error_id = new_error_id()
324                 request.state.error_id = error_id
325             except Exception:
326                 error_id = None
327         if exc.status_code >= 500:
328             try:
329                 from app.core.observability.aols import LogEvent, get_logger
330                 get_logger("app.errors").error(
331                     str(LogEvent.SYSTEM_ERROR),
332                     error_type=type(exc).__name__,
333                     error_message=exc.message,
334                     status_code=exc.status_code,
335                     path=request.url.path,
336                     request_id=rid,
337                     error_id=error_id,
338                 )
339             except Exception:
340                 pass
341         body = error_response(
342             exc.status_code,
343             exc.message,
344             exc.detail,
345             request_id=rid,
346             error_id=error_id if exc.status_code >= 500 else None,
347         )
348         resp = JSONResponse(status_code=exc.status_code, content=body)
349         if error_id:
350             resp.headers["X-Error-ID"] = str(error_id)
351         return resp
352     @app.exception_handler(Exception)

```

```

361 async def global_exception_handler(request: Request, exc: Exception) -> JSONResponse:
362     import traceback
363     rid = getattr(request.state, "request_id", None)
364     try:
365         from app.core.observability.aols import LogEvent, get_logger, new_error_id
366         error_id = getattr(request.state, "error_id", None) or new_error_id()
367         request.state.error_id = error_id
368         get_logger("app.errors").error(
369             str(LogEvent.SYSTEM_ERROR),
370             error_type=type(exc).__name__,
371             error_message=str(exc),
372             status_code=500,
373             path=request.url.path,
374             request_id=rid,
375             error_id=error_id,
376             exc_info=True,
377         )
378     except Exception:
379         error_id = getattr(request.state, "error_id", None)
380         stack = traceback.format_exc() if settings.DEBUG else None
381         resp = JSONResponse(
382             status_code=500,
383             content=error_response(
384                 500,
385                 "Internal error",
386                 str(exc) if settings.DEBUG else None,
387                 request_id=rid,
388                 stacktrace=stack,
389                 error_id=error_id,
390             ),
391         )
392         if error_id:
393             resp.headers["X-Error-ID"] = str(error_id)
394         return resp
395 @app.get("/metrics", include_in_schema=False)
396 async def prometheus_metrics() -> Any:
397     """Prometheus text exposition endpoint (scrapped by Prometheus / Grafana Agent).
398
399     Public path (no API key). Disable via ``METRICS_ENABLED=false`` to return 404.
400     """
401     if not settings.METRICS_ENABLED:
402         return JSONResponse(status_code=404, content={"detail": "Metrics disabled"})
403     return get_metrics_response()
404 APP_VERSION = "1.0.0"
405 async def _probe_services() -> dict[str, str]:
406     """Probe database and Redis; return per-service status strings."""
407     from sqlalchemy import text
408     from app.core.dependencies import engine
409     services: dict[str, str] = {}
410     try:
411         async with engine.connect() as conn:
412             await conn.execute(text("SELECT 1"))
413             services["database"] = "ok"
414     except Exception as e:
415         services["database"] = f"error: {e}"

```

```

433 skip_redis = bool(settings.CELERY_TASK_ALWAYS_EAGER)
434 try:
435     from app.core.profiles import current_profile, get_task_queue
436     if current_profile() == "lite" or get_task_queue().is_eager():
437         skip_redis = True
438     if get_task_queue().backend_name in {"eager", "memory"}:
439         skip_redis = True
440 except Exception:
441     pass
442 if skip_redis:
443     services["redis"] = "skipped (lite/eager)"
444 else:
445     try:
446         from app.core.dependencies import get_redis
447         r = await get_redis()
448         await r.ping()
449         services["redis"] = "ok"
450     except Exception:
451         services["redis"] = "unavailable"
452 return services
453 @app.get("/health/live")
454 async def liveness() -> dict[str, Any]:
455     """Kubernetes-style liveness probe – process is up, no dependency checks.

457     Always returns HTTP 200 if the event loop can serve the request.
458     """
459     return {
460         "status": "alive",
461         "app": settings.APP_NAME,
462         "version": APP_VERSION,
463     }
464 @app.get("/health/ready")
465 async def readiness(request: Request) -> JSONResponse:
466     """Kubernetes-style readiness probe – ready to accept traffic.

468     Returns 200 when the database is reachable; 503 when critical deps fail.
469     Redis is required only when not in Celery eager mode.
470     """
471     services = await _probe_services()
472     critical_ok = services.get("database") == "ok"
473     redis_ok = services.get("redis", "").startswith(("ok", "skipped"))
474     ready = critical_ok and redis_ok
475     deploy = {}
476     try:
477         from app.core.profiles import profile_status
478         deploy = profile_status()
479     except Exception:
480         pass
481     body: dict[str, Any] = {
482         "status": "ready" if ready else "not_ready",
483         "app": settings.APP_NAME,
484         "version": APP_VERSION,
485         "services": services,
486         "celery_eager": settings.CELERY_TASK_ALWAYS_EAGER,
487         "deploy": deploy,
488     }
489     return JSONResponse(status_code=200 if ready else 503, content=body)
490 @app.get("/health")

```

```

505 async def health_check() -> dict[str, Any]:
506     """Composite health check with database and Redis connectivity.

508     Backward-compatible with existing Docker healthchecks. Prefer
509     ``/health/live`` and ``/health/ready`` for orchestrators.
510     """
511     services = await _probe_services()
512     critical_ok = services.get("database") == "ok"
513     redis_ok = services.get("redis", "").startswith(("ok", "skipped"))
514     all_healthy = critical_ok and redis_ok
515     deploy = {}
516     try:
517         from app.core.profiles import profile_status
518         deploy = profile_status()
519     except Exception:
520         pass
521     return {
522         "status": "healthy" if all_healthy else "degraded",
523         "app": settings.APP_NAME,
524         "version": APP_VERSION,
525         "services": services,
526         "celery_eager": settings.CELERY_TASK_ALWAYS_EAGER,
527         "deploy": deploy,
528     }
529 # === File: app/config.py ===
530 """Application configuration using pydantic-settings with environment modes."""
531 from __future__ import annotations
532 from pathlib import Path
533 from typing import Literal
534 from pydantic_settings import BaseSettings, SettingsConfigDict
535 class Settings(BaseSettings):
536     """Production-grade configuration with dev/test/prod separation.

538     Environment variables override .env values which override defaults.
539     """
540     ENV: Literal["dev", "test", "prod"] = "dev"
541     DEBUG: bool = True
542     APP_NAME: str = "AgentFlow Intelligence"
543     API_V1_PREFIX: str = "/api/v1"
544     SECRET_KEY: str = "change-me-in-production"
545     CORS_ORIGINS: list[str] = [
546         "http://localhost:5173",
547         "http://127.0.0.1:5173",
548         "http://localhost:3000",
549     ]
550     DATABASE_URL: str = "sqlite+aiosqlite:///./agentflow_eval.db"
551     DB_POOL_SIZE: int = 20
552     DB_MAX_OVERFLOW: int = 40
553     DB_ECHO: bool = False
554     REDIS_URL: str = "redis://localhost:6379/0"
555     CELERY_BROKER_URL: str = "redis://localhost:6379/0"
556     CELERY_RESULT_BACKEND: str = "redis://localhost:6379/0"
557     CELERY_TASK_ALWAYS_EAGER: bool = False
558     OPENAI_API_KEY: str = ""
559     OPENAI_BASE_URL: str = "https://api.openai.com/v1"

```



```
577 LOG_LEVEL: str = "INFO"
578 LOG_FILE: str = "logs/agentflow_eval.log"
579 LOG_FORMAT: Literal["json", "text"] = "text"
580 LOG_MAX_BYTES: int = 10485760
581 LOG_BACKUP_COUNT: int = 30
582 LOG_SERVICE_NAME: str = "agentflow-api"
583 LOG_ACCESS_SKIP_PATHS: str = "/health,/metrics,/favicon.ico"
584 LOG_REDACT_MAX_STR: int = 2000
585 LOG_DB_SINK: bool = True
586 LOG_DB_BATCH_SIZE: int = 32
587 LOG_DB_FLUSH_INTERVAL_SEC: float = 1.0
588 LOG_DB_RETENTION_DAYS: int = 30
589 RATE_LIMIT_ENABLED: bool = True
590 RATE_LIMIT_DEFAULT: str = "100/minute"
591 AUTH_ENABLED: bool = False
592 API_KEYS: str = ""
593 RBAC_ENABLED: bool = True
594 DEFAULT_ROLE: str = "member"
595 ACTOR_ROLES: str = ""
596 TENANCY_ENABLED: bool = False
597 ADMIN_ACTORS: str = "admin"
598 MULTI_TENANT_ENABLED: bool = False
599 TOOL_TIMEOUT_SEC: float = 3.0
600 HTTP_AGENT_ALLOW_PRIVATE_IP: bool = False
601 HTTP_AGENT_DEFAULT_TIMEOUT_SEC: float = 60.0
602 HTTP_AGENT_SSRF_RESOLVE_DNS: bool = True
603 MAX_REQUEST_SIZE: int = 10 * 1024 * 1024
604 DEFAULT_PAGE_SIZE: int = 20
605 MAX_PAGE_SIZE: int = 100
606 GUNICORN_WORKERS: int = 4
607 GUNICORN_WORKER_CLASS: str = "uvicorn.workers.UvicornWorker"
608 GUNICORN_TIMEOUT: int = 120
609 JUDGE_TIMEOUT_SEC: float = 60.0
610 JUDGE_CACHE_SIZE: int = 128
611 METRICS_ENABLED: bool = True
612 CACHE_ENABLED: bool = True
613 CACHE_WARMUP_ON_STARTUP: bool = False
614 LLM_RETRY_ENABLED: bool = True
615 LLM_MAX_RETRIES: int = 3
616 LLM_RETRY_MIN_WAIT_SEC: float = 1.0
617 LLM_RETRY_MAX_WAIT_SEC: float = 10.0
618 LLM_CALL_TIMEOUT_SEC: float = 30.0
619 CIRCUIT_ENABLED: bool = True
620 CIRCUIT_FAILURE_THRESHOLD: int = 5
621 CIRCUIT_RECOVERY_TIMEOUT_SEC: float = 60.0
622 STORAGE_BACKEND: str = "local"
623 LOCAL_STORAGE_PATH: str = "data/uploads"
624 S3_BUCKET: str = "agentflow"
625 S3_ENDPOINT_URL: str = ""
626 S3_ACCESS_KEY: str = ""
627 S3_SECRET_KEY: str = ""
628 S3_REGION: str = "us-east-1"
629 S3_PREFIX: str = ""
630 MEDIA_MAX_UPLOAD_BYTES: int = 20 * 1024 * 1024
631 VISION_MODEL: str = "gpt-4o-mini"
```

```

649 DEPLOY_PROFILE: str = "auto"
650 TASK_QUEUE_BACKEND: str = ""
651 BILLING_ENABLED: bool = False
652 STRIPE_MODE: str = "mock"
653 STRIPE_SECRET_KEY: str = ""
654 STRIPE_WEBHOOK_SECRET: str = ""
655 STRIPE_SUCCESS_URL: str = "http://localhost:5173/billing?checkout=success"
656 STRIPE_CANCEL_URL: str = "http://localhost:5173/billing?checkout=cancel"
657 STRIPE_PRICE_IDS: str = ""
658 SLOW_TASK_THRESHOLD_SEC: float = 30.0
659 OBSERVABILITY_KPI_ENABLED: bool = True
660 PLUGINS_ENABLED: bool = True
661 PLUGIN_DIRS: str = "plugins"
662 PLUGIN_MODULES: str = ""
663 PLUGIN_CATALOG_PATH: str = ""
664 PLUGIN_MARKET_SEED_EXAMPLES: bool = True
665 PLUGIN_STRICT_ALLOWLIST: bool = False
666 PLUGIN_STRICT_MODE: bool = False
667 PLUGIN_ALLOWLIST: str = ""
668 PLUGIN_SIGNATURE_CHECK: bool = False
669 PLUGIN_SIGNING_SECRET: str = ""
670 PLUGIN_SIGNATURES: str = ""
671 @property
672 def plugin_dir_list(self) -> list[str]:
673     raw = (self.PLUGIN_DIRS or "").strip()
674     if not raw:
675         return []
676     return [p.strip() for p in raw.replace(";", ",").split(",") if p.strip()]
677 @property
678 def plugin_module_list(self) -> list[str]:
679     raw = (self.PLUGIN_MODULES or "").strip()
680     if not raw:
681         return []
682     return [p.strip() for p in raw.replace(";", ",").split(",") if p.strip()]
683 @property
684 def plugin_allowlist(self) -> list[str]:
685     raw = (self.PLUGIN_ALLOWLIST or "").strip()
686     if not raw:
687         return []
688     return [p.strip() for p in raw.replace(";", ",").split(",") if p.strip()]
689 @property
690 def is_dev(self) -> bool:
691     return self.ENV == "dev"
692 @property
693 def is_test(self) -> bool:
694     return self.ENV == "test"
695 @property
696 def is_prod(self) -> bool:
697     return self.ENV == "prod"
698 model_config = SettingsConfigDict(
699     env_file=".env",
700     env_file_encoding="utf-8",
701     extra="ignore",
702     case_sensitive=True,
703 )

```

```

721 settings = Settings()
722 Path(settings.LOG_FILE).parent.mkdir(parents=True, exist_ok=True)
723 # === File: app/core/dependencies.py ===
724 """Dependency injection with database pool and Redis cache."""
725 from __future__ import annotations
726 from collections.abc import AsyncGenerator
727 from typing import Any
728 import redis.asyncio as aioredis
729 from sqlalchemy.ext.asyncio import (
730     AsyncSession,
731     async_sessionmaker,
732     create_async_engine,
733 )
734 from app.config import settings
735 _engine_kw: dict = {
736     "echo": settings.DB_ECHO,
737     "future": True,
738 }
739 db_url = settings.DATABASE_URL or ""
740 if "postgresql" in db_url:
741     _engine_kw.update(
742         {
743             "pool_size": settings.DB_POOL_SIZE,
744             "max_overflow": settings.DB_MAX_OVERFLOW,
745             "pool_pre_ping": True,
746             "pool_recycle": 3600,
747         }
748     )
749 elif "sqlite" in db_url:
750     _engine_kw.update(
751         {
752             "connect_args": {"check_same_thread": False},
753         }
754     )
755 engine = create_async_engine(settings.DATABASE_URL, **_engine_kw)
756 if "sqlite" in db_url:
757     from sqlalchemy import event
758     @event.listens_for(engine.sync_engine, "connect")
759     def _sqlite_on_connect(dbapi_conn, connection_record) -> None:
760         cursor = dbapi_conn.cursor()
761         try:
762             cursor.execute("PRAGMA journal_mode=WAL")
763             cursor.execute("PRAGMA synchronous=NORMAL")
764             cursor.execute("PRAGMA foreign_keys=ON")
765             cursor.execute("PRAGMA temp_store=MEMORY")
766             cursor.execute("PRAGMA busy_timeout=5000")
767         finally:
768             cursor.close()
769 async_session_factory = async_sessionmaker(
770     engine,
771     class_=AsyncSession,
772     expire_on_commit=False,
773 )

```

```

793 async def get_db() -> AsyncGenerator[AsyncSession, Any]:
794     """Provide an async database session with auto-commit/rollback."""
795     async with async_session_factory() as session:
796         try:
797             yield session
798             await session.commit()
799         except Exception:
800             await session.rollback()
801             raise
802         finally:
803             await session.close()
804 async def bind_tenant(request: Any, session: AsyncSession) -> Any:
805     """Resolve X-Tenant-ID against DB membership; store on request.state.

807     Call from endpoints after injecting ``Request`` + ``get_db`` session, or
808     wrap with ``Depends`` via a thin FastAPI-layer helper in routers.
809     """
810     from app.core.tenant_context import extract_tenant_header, resolve_tenant_context
811     actor = getattr(request.state, "actor", None) or "anonymous"
812     role = getattr(request.state, "role", None)
813     role_s = role.value if hasattr(role, "value") else (str(role) if role else "")
814     header = extract_tenant_header(request)
815     ctx = await resolve_tenant_context(
816         session,
817         actor=actor,
818         header_value=header,
819         system_role=role_s,
820     )
821     request.state.tenant_id = ctx.tenant_id
822     request.state.tenant_slug = ctx.tenant_slug
823     request.state.tenant_member_role = ctx.member_role
824     return ctx
825 _redis: aioredis.Redis | None = None
826 async def get_redis() -> aioredis.Redis:
827     """Lazy-init Redis connection pool."""
828     global _redis
829     if _redis is None:
830         _redis = aioredis.from_url(
831             settings.REDIS_URL,
832             encoding="utf-8",
833             decode_responses=True,
834             socket_timeout=0.8,
835             socket_connect_timeout=0.5,
836         )
837     return _redis
838 async def close_redis() -> None:
839     """Close the Redis connection pool gracefully."""
840     global _redis
841     if _redis:
842         await _redis.close()
843     _redis = None
844 CACHE_TTL_DEFAULT = 300
845 CACHE_TTL_LIST = 60
846 async def cache_get(key: str) -> Any | None:
847     """Get value from multi-layer cache (L1 + Redis)."""
848     from app.core.cache.client import get_cache
849     return await get_cache().get(key)

```

```

865 async def cache_set(key: str, value: Any, ttl: int = CACHE_TTL_DEFAULT) -> None:
866     """Set value in multi-layer cache with TTL."""
867     from app.core.cache.client import get_cache
868     await get_cache().set(key, value, ttl=ttl)
869 async def cache_invalidate(pattern: str) -> None:
870     """Invalidate all keys matching a pattern."""
871     from app.core.cache.client import get_cache
872     await get_cache().delete_pattern(pattern)
873 # === File: app/core/events.py ===
874 """Task activity events – Redis pub/sub + local callback bridge for WebSocket.

876 Celery workers publish here; the API process subscribes and fans out to WS clients.
877 When Redis is unavailable, in-process listeners still receive local publishes.
878 """
879 from __future__ import annotations
880 import json
881 import logging
882 from datetime import datetime, timezone
883 from typing import Any, Awaitable, Callable
884 logger = logging.getLogger(__name__)
885 CHANNEL = "agentflow:task_events"
886 _local_listeners: list[Callable[[dict[str, Any]], Awaitable[None]]] = []
887 def register_local_listener(cb: Callable[[dict[str, Any]], Awaitable[None]]) -> None:
888     if cb not in _local_listeners:
889         _local_listeners.append(cb)
890 def unregister_local_listener(cb: Callable[[dict[str, Any]], Awaitable[None]]) -> None:
891     if cb in _local_listeners:
892         _local_listeners.remove(cb)
893 def build_task_event(
894     *,
895     task_id: str,
896     task_name: str,
897     status: str,
898     prev_status: str | None = None,
899     actor: str | None = None,
900     extra: dict[str, Any] | None = None,
901 ) -> dict[str, Any]:
902     payload: dict[str, Any] = {
903         "type": "task_status",
904         "task_id": task_id,
905         "task_name": task_name,
906         "status": status,
907         "prev_status": prev_status,
908         "actor": actor or "anonymous",
909         "at": datetime.now(timezone.utc).isoformat(),
910     }
911     if extra:
912         payload["extra"] = extra
913     return payload
914 def publish_task_event(event: dict[str, Any]) -> None:
915     """Sync-safe publish for Celery workers and FastAPI endpoints.

917     1) Publish to Redis (best-effort)
918     2) Schedule local async listeners if a loop is running
919     """
920     body = json.dumps(event, ensure_ascii=False, default=str)
921     try:
922         import redis as sync_redis
923         from app.config import settings
924         client = sync_redis.from_url(settings.REDIS_URL, socket_connect_timeout=0.5)
925         try:
926             client.publish(CHANNEL, body)
927         finally:
928             client.close()
929     except Exception as exc:
930         try:
931             import asyncio
932             loop = asyncio.get_running_loop()
933         except RuntimeError:
934             return
935     for cb in list(_local_listeners):
936         try:
937             loop.create_task(cb(event))
938         except Exception as exc:

```

```
937 def publish_task_status(  
938     task_id: str,  
939     task_name: str,  
940     status: str,  
941     *,  
942     prev_status: str | None = None,  
943     actor: str | None = None,  
944 ) -> None:  
945     publish_task_event(  
946         build_task_event(  
947             task_id=task_id,  
948             task_name=task_name,  
949             status=status if isinstance(status, str) else str(status),  
950             prev_status=prev_status,  
951             actor=actor,  
952         )  
953     )  
954 # === File: app/core/middleware.py ===  
955 """Request tracing, security headers, and optional API-key auth middleware."""  
956 from __future__ import annotations  
957 import time  
958 import uuid  
959 from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint  
960 from starlette.requests import Request  
961 from starlette.responses import JSONResponse, Response  
962 from app.config import settings  
963 from app.core.security import (  
964     authenticate_api_key,  
965     extract_api_key,  
966     is_public_path,  
967 )  
968 from app.utils.exceptions import error_response  
969 try:  
970     from app.core.observability.aols import (  
971         LogEvent,  
972         bind_request_context,  
973         clear_request_context,  
974         get_logger,  
975         new_error_id,  
976     )  
977     log = get_logger("app.http")  
978     HAS_AOLS = True  
979 except Exception:  
980     import logging  
981     log = logging.getLogger(__name__)  
982     HAS_AOLS = False  
983     LogEvent = None  
984     def bind_request_context(**kwargs):  
985         return None  
986     def clear_request_context() -> None:  
987         return None  
988     def new_error_id() -> str:  
989         return uuid.uuid4().hex[:16]
```

```

1009 def _client_ip(request: Request) -> str:
1010     forwarded = request.headers.get("X-Forwarded-For")
1011     if forwarded:
1012         return forwarded.split(",")[0].strip()
1013     if request.client:
1014         return request.client.host or ""
1015     return ""
1016 def _skip_access_log(path: str) -> bool:
1017     raw = getattr(settings, "LOG_ACCESS_SKIP_PATHS", "") or ""
1018     prefixes = [p.strip() for p in raw.split(",") if p.strip()]
1019     for p in prefixes:
1020         if path == p or path.startswith(p.rstrip("/") + "/") or path.startswith(p):
1021             if path == p or path.startswith(p):
1022                 return True
1023     return False
1024 class RequestIDMiddleware(BaseHTTPMiddleware):
1025     """Inject request/trace IDs, bind AOLS context, emit structured access logs.
1026
1027     Behaviour:
1028     1. Reads ``X-Request-ID`` / ``X-Trace-ID`` or generates a UUID.
1029     2. Sets request.state + contextvars TraceID + structlog bind.
1030     3. Echoes IDs on response; optional ``X-Error-ID`` when set.
1031     4. Logs each request as ``http.request`` / ``http.request.failed``.
1032     """
1033     async def dispatch(
1034         self, request: Request, call_next: RequestResponseEndpoint
1035     ) -> Response:
1036         request_id = (
1037             request.headers.get("X-Request-ID")
1038             or request.headers.get("X-Trace-ID")
1039             or str(uuid.uuid4())
1040         )
1041         request.state.request_id = request_id
1042         try:
1043             from app.core.observability.tracing import set_trace_id
1044             set_trace_id(request_id)
1045         except Exception:
1046             pass
1047         client_ip = _client_ip(request)
1048         path = request.url.path
1049         method = request.method
1050         bind_request_context(
1051             request_id=request_id,
1052             trace_id=request_id,
1053             method=method,
1054             path=path,
1055             client_ip=client_ip,
1056         )
1057         start = time.perf_counter()
1058         error_id: str | None = None
1059         status_code = 500
1060         response: Response | None = None
1061         try:
1062             response = await call_next(request)
1063             status_code = response.status_code

```

```

1081     except Exception:
1082         error_id = new_error_id()
1083         request.state.error_id = error_id
1084         duration_ms = int((time.perf_counter() - start) * 1000)
1085         if not _skip_access_log(path):
1086             self._emit_access(
1087                 method=method,
1088                 path=path,
1089                 status_code=500,
1090                 latency_ms=duration_ms,
1091                 request_id=request_id,
1092                 client_ip=client_ip,
1093                 error_id=error_id,
1094                 failed=True,
1095             )
1096         clear_request_context()
1097         raise
1098     actor = getattr(request.state, "actor", None)
1099     if actor:
1100         bind_request_context(actor=str(actor))
1101     duration_ms = int((time.perf_counter() - start) * 1000)
1102     response.headers["X-Request-ID"] = request_id
1103     response.headers["X-Trace-ID"] = request_id
1104     err_state = getattr(request.state, "error_id", None)
1105     if err_state:
1106         response.headers["X-Error-ID"] = str(err_state)
1107         error_id = str(err_state)
1108     failed = status_code >= 400
1109     if not _skip_access_log(path):
1110         self._emit_access(
1111             method=method,
1112             path=path,
1113             status_code=status_code,
1114             latency_ms=duration_ms,
1115             request_id=request_id,
1116             client_ip=client_ip,
1117             actor=str(actor) if actor else None,
1118             error_id=error_id,
1119             failed=failed,
1120         )
1121     clear_request_context()
1122     return response
1123 @staticmethod
1124 def _emit_access(
1125     *,
1126     method: str,
1127     path: str,
1128     status_code: int,
1129     latency_ms: int,
1130     request_id: str,
1131     client_ip: str,
1132     actor: str | None = None,
1133     error_id: str | None = None,
1134     failed: bool = False,
1135 ) -> None:

```



```

1153     event = (
1154         str(LogEvent.HTTP_REQUEST_FAILED)
1155         if failed and LogEvent is not None
1156         else (
1157             str(LogEvent.HTTP_REQUEST) if LogEvent is not None else "http.request"
1158         )
1159     )
1160     if failed and LogEvent is None:
1161         event = "http.request.failed"
1162     fields: dict = {
1163         "event": event,
1164         "method": method,
1165         "path": path,
1166         "status_code": status_code,
1167         "latency_ms": latency_ms,
1168         "request_id": request_id,
1169         "trace_id": request_id,
1170         "client_ip": client_ip,
1171     }
1172     if actor:
1173         fields["actor"] = actor
1174     if error_id:
1175         fields["error_id"] = error_id
1176     try:
1177         if failed:
1178             log.warning(event, **{k: v for k, v in fields.items() if k != "event"})
1179         else:
1180             log.info(event, **{k: v for k, v in fields.items() if k != "event"})
1181     except TypeError:
1182         log.info(
1183             "%s %s -> %s [%dms] request_id=%s",
1184             method,
1185             path,
1186             status_code,
1187             latency_ms,
1188             request_id,
1189         )
1190     class SecurityHeadersMiddleware(BaseHTTPMiddleware):
1191         """Attach enterprise security response headers to every response."""
1192         _API_CSP = "default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none';"
1193         _DOCS_CSP = (
1194             "default-src 'self'; "
1195             "script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net; "
1196             "style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net; "
1197             "img-src 'self' data: https:; "
1198             "font-src 'self' https://cdn.jsdelivr.net data:; "
1199             "connect-src 'self'; "
1200             "frame-ancestors 'none'"
1201         )
1202     async def dispatch(
1203         self, request: Request, call_next: RequestResponseEndpoint
1204     ) -> Response:
1205         response = await call_next(request)
1206         response.headers.setdefault("X-Content-Type-Options", "nosniff")
1207         response.headers.setdefault("X-Frame-Options", "DENY")

```

```

1225     response.headers.setdefault(
1226         "Referrer-Policy", "strict-origin-when-cross-origin"
1227     )
1228     response.headers.setdefault(
1229         "Permissions-Policy",
1230         "geolocation=(), microphone=(), camera=(), payment=()",
1231     )
1232     response.headers.setdefault("X-XSS-Protection", "0")
1233     response.headers.setdefault("Cross-Origin-Opener-Policy", "same-origin")
1234     response.headers.setdefault("Cross-Origin-Resource-Policy", "same-site")
1235     path = request.url.path
1236     if path in {"/docs", "/redoc", "/openapi.json"} or path.startswith("/docs"):
1237         response.headers.setdefault("Content-Security-Policy", self._DOCS_CSP)
1238     else:
1239         response.headers.setdefault("Content-Security-Policy", self._API_CSP)
1240     if settings.is_prod and not settings.DEBUG:
1241         response.headers.setdefault(
1242             "Strict-Transport-Security",
1243             "max-age=31536000; includeSubDomains",
1244         )
1245     return response
1246 class APIKeyAuthMiddleware(BaseHTTPMiddleware):
1247     """Enforce API key auth for /api/v1 when AUTH_ENABLED=true."""
1248     async def dispatch(
1249         self, request: Request, call_next: RequestResponseEndpoint
1250     ) -> Response:
1251         path = request.url.path
1252         if not settings.AUTH_ENABLED:
1253             from app.core.rbac import Role
1254             request.state.actor = "anonymous"
1255             request.state.role = Role.SYSTEM_ADMIN
1256             return await call_next(request)
1257         if is_public_path(path) or not path.startswith("/api/"):
1258             from app.core.rbac import Role
1259             request.state.actor = "public"
1260             request.state.role = Role.VIEWER
1261             return await call_next(request)
1262         if request.method == "OPTIONS":
1263             return await call_next(request)
1264         api_key = extract_api_key(request)
1265         identity = authenticate_api_key(api_key)
1266         if identity is None:
1267             rid = getattr(request.state, "request_id", None)
1268             return JSONResponse(
1269                 status_code=401,
1270                 content=error_response(
1271                     401,
1272                     "Unauthorized",
1273                     "Provide a valid X-API-Key or Authorization: Bearer <key>",
1274                     request_id=rid,
1275                 ),
1276             )
1277         request.state.actor = identity.actor
1278         request.state.auth = identity
1279         request.state.role = identity.role
1280         return await call_next(request)
1281 # === File: app/core/security.py ===
1282 """API Key authentication helpers with optional RBAC role binding."""
1283 from __future__ import annotations
1284 import hmac
1285 import secrets
1286 from dataclasses import dataclass
1287 from fastapi import Header, Request
1288 from fastapi.security.utils import get_authorization_scheme_param
1289 from app.config import settings
1290 from app.core.rbac import Role, resolve_role_for_actor
1291 from app.utils.exceptions import AgentFlowError

```

```

1297 class UnauthorizedError(AgentFlowError):
1298     """Raised when authentication fails."""
1299     def __init__(self, message: str = "Unauthorized") -> None:
1300         super().__init__(message=message, status_code=401, detail=None)
1301 @dataclass(frozen=True)
1302 class ApiKeyEntry:
1303     """Parsed API key configuration entry."""
1304     secret: str
1305     actor: str
1306     role: Role | None = None
1307 @dataclass(frozen=True)
1308 class AuthIdentity:
1309     """Authenticated principal derived from API key."""
1310     key_id: str
1311     actor: str
1312     raw_key_prefix: str
1313     role: Role = Role.USER
1314 def parse_api_key_entries(raw: str | list[str] | None) -> list[ApiKeyEntry]:
1315     """Parse API_KEYS into structured entries.

1317     Supported formats per item:
1318     - ``secret``
1319     - ``secret:actor``
1320     - ``secret:actor:role`` (role is admin|manager|reviewer|user|guest)
1321     """
1322     if not raw:
1323         return []
1324     items: list[str]
1325     if isinstance(raw, list):
1326         items = [str(x).strip() for x in raw if str(x).strip()]
1327     else:
1328         items = [p.strip() for p in str(raw).split(",") if p.strip()]
1329     entries: list[ApiKeyEntry] = []
1330     for idx, item in enumerate(items, start=1):
1331         parts = [p.strip() for p in item.split(":")]
1332         secret = parts[0] if parts else ""
1333         if not secret:
1334             continue
1335         actor = parts[1] if len(parts) > 1 and parts[1] else f"key_{idx}"
1336         role: Role | None = None
1337         if len(parts) > 2 and parts[2]:
1338             try:
1339                 role = Role.parse(parts[2])
1340             except ValueError:
1341                 actor = f"{actor}:{parts[2]}"
1342                 role = None
1343         entries.append(ApiKeyEntry(secret=secret, actor=actor, role=role))
1344     return entries
1345 def parse_api_keys(raw: str | list[str] | None) -> dict[str, str]:
1346     """Parse API keys config into {secret: actor_name}.

1348     Backward-compatible helper used by settings UI and tests.
1349     """
1350     return {e.secret: e.actor for e in parse_api_key_entries(raw)}
1351 def extract_api_key(
1352     request: Request,
1353     authorization: str | None = None,
1354     x_api_key: str | None = None,
1355 ) -> str | None:
1356     """Extract API key from X-API-Key or Authorization: Bearer."""
1357     key = (x_api_key or request.headers.get("X-API-Key") or "").strip()
1358     if key:
1359         return key
1360     header = authorization or request.headers.get("Authorization")
1361     if header:
1362         scheme, param = get_authorization_scheme_param(header)
1363         if scheme.lower() == "bearer" and param:
1364             return param.strip()
1365         if scheme.lower() in {"apikey", "api-key"} and param:
1366             return param.strip()
1367     return None

```

```

1369 def authenticate_api_key(api_key: str | None) -> AuthIdentity | None:
1370     """Validate API key against configured secrets (constant-time compare)."""
1371     if not api_key:
1372         return None
1373     entries = parse_api_key_entries(settings.API_KEYS)
1374     for entry in entries:
1375         if hmac.compare_digest(api_key, entry.secret):
1376             role = resolve_role_for_actor(entry.actor, explicit=entry.role)
1377             return AuthIdentity(
1378                 key_id=secrets.token_hex(4),
1379                 actor=entry.actor,
1380                 raw_key_prefix=api_key[:4] + "****",
1381                 role=role,
1382             )
1383     return None
1384 def require_auth_if_enabled(
1385     request: Request,
1386     authorization: str | None = Header(default=None),
1387     x_api_key: str | None = Header(default=None, alias="X-API-Key"),
1388 ) -> AuthIdentity | None:
1389     """FastAPI dependency: enforce API key when AUTH_ENABLED is true."""
1390     if not settings.AUTH_ENABLED:
1391         identity = AuthIdentity(
1392             key_id="dev",
1393             actor="anonymous",
1394             raw_key_prefix="dev",
1395             role=Role.SYSTEM_ADMIN,
1396         )
1397         request.state.actor = identity.actor
1398         request.state.auth = identity
1399         request.state.role = identity.role
1400         return identity
1401     api_key = extract_api_key(request, authorization, x_api_key)
1402     identity = authenticate_api_key(api_key)
1403     if identity is None:
1404         raise UnauthorizedError(
1405             "Invalid or missing API key. Provide X-API-Key or Authorization: Bearer <key>."
1406         )
1407     request.state.actor = identity.actor
1408     request.state.auth = identity
1409     request.state.role = identity.role
1410     return identity
1411 AUTH_PUBLIC_PATHS = frozenset(
1412     {
1413         "/health",
1414         "/health/live",
1415         "/health/ready",
1416         "/metrics",
1417         "/docs",
1418         "/redoc",
1419         "/openapi.json",
1420         "/api/v1/billing/webhook/stripe",
1421         "/api/v1/billing/webhook",
1422     }
1423 )

```

```

1441 def is_public_path(path: str) -> bool:
1442     """Return True if the path is exempt from API key authentication."""
1443     if path in AUTH_PUBLIC_PATHS:
1444         return True
1445     if path.startswith("/docs") or path.startswith("/redoc"):
1446         return True
1447     if path.rstrip("/").endswith("/billing/webhook/stripe"):
1448         return True
1449     if path.rstrip("/").endswith("/billing/webhook"):
1450         return True
1451     return False
1452 # === File: app/core/settings_guard.py ===
1453 """Production settings validation – fail fast on insecure prod config."""
1454 from __future__ import annotations
1455 import logging
1456 import re
1457 from dataclasses import dataclass, field
1458 from typing import TYPE_CHECKING
1459 if TYPE_CHECKING:
1460     from app.config import Settings
1461 logger = logging.getLogger(__name__)
1462 _INSECURE_SECRET_KEYS = frozenset(
1463     {
1464         "change-me-in-production",
1465         "secret",
1466         "changeme",
1467         "dev",
1468         "development",
1469         "test",
1470         "password",
1471         "",
1472     }
1473 )
1474 _WEAK_SECRET_RE = re.compile(r"^(.)\{7,$}")
1475 class ProductionConfigError(RuntimeError):
1476     """Raised when production configuration fails hard security checks."""
1477     def __init__(self, errors: list[str]) -> None:
1478         self.errors = list(errors)
1479         message = "Production configuration invalid:\n - " + "\n - ".join(self.errors)
1480         super().__init__(message)
1481 @dataclass
1482 class SettingsValidationResult:
1483     """Outcome of validating application settings for a target environment."""
1484     errors: list[str] = field(default_factory=list)
1485     warnings: list[str] = field(default_factory=list)
1486     @property
1487     def ok(self) -> bool:
1488         """True when no hard errors were found."""
1489         return not self.errors
1490 def validate_settings(
1491     settings: Settings,
1492     *,
1493     strict: bool | None = None,
1494 ) -> SettingsValidationResult:
1495     """Validate settings for the current (or forced) environment.

```

```

1514     Args:
1515         settings: Application settings instance.
1516         strict: When True, apply production rules. When None, use ``settings.is_prod``.

1518     Returns:
1519         SettingsValidationResult with ``errors`` (must fix) and ``warnings`` (should fix).
1520     """
1521     result = SettingsValidationResult()
1522     use_strict = settings.is_prod if strict is None else strict
1523     if not use_strict:
1524         if settings.AUTH_ENABLED and not str(settings.API_KEYS or "").strip():
1525             result.errors.append(
1526                 "AUTH_ENABLED=true but API_KEYS is empty – authentication would lock out a
1527             )
1528         return result
1529     secret = (settings.SECRET_KEY or "").strip()
1530     if secret.lower() in _INSECURE_SECRET_KEYS or len(secret) < 16:
1531         result.errors.append(
1532             "SECRET_KEY must be a strong random string (≥16 chars); "
1533             "do not use the default 'change-me-in-production'"
1534         )
1535     elif _WEAK_SECRET_RE.match(secret):
1536         result.errors.append("SECRET_KEY looks weak (repeated single character)")
1537     if settings.DEBUG:
1538         result.errors.append("DEBUG must be false when ENV=prod (leaks stacktraces)")
1539     if settings.AUTH_ENABLED and not str(settings.API_KEYS or "").strip():
1540         result.errors.append("AUTH_ENABLED=true requires a non-empty API_KEYS list")
1541     if not settings.AUTH_ENABLED:
1542         result.warnings.append(
1543             "AUTH_ENABLED=false in production – API is publicly reachable; "
1544             "enable AUTH_ENABLED and set API_KEYS for enterprise deployments"
1545         )
1546     db_url = (settings.DATABASE_URL or "").lower()
1547     if db_url.startswith("sqlite"):
1548         result.warnings.append(
1549             "DATABASE_URL uses SQLite in production – prefer PostgreSQL for concurrency an
1550         )
1551     if settings.LOG_FORMAT != "json":
1552         result.warnings.append(
1553             "LOG_FORMAT is not 'json' – structured JSON logs are recommended for productio
1554         )
1555     cors = settings.CORS_ORIGINS or []
1556     if "*" in cors:
1557         result.warnings.append(
1558             "CORS_ORIGINS contains '*' – restrict origins to known frontends in production
1559         )
1560     if not cors:
1561         result.warnings.append("CORS_ORIGINS is empty – browser clients may be blocked")
1562     return result
1563 def enforce_production_settings(settings: Settings) -> SettingsValidationResult:
1564     """Run validation and raise ProductionConfigError on hard failures.

1566     Always logs warnings. Only enforces errors when ``settings.is_prod``.

```

```

1585     Args:
1586         settings: Application settings instance.

1588     Returns:
1589         The validation result (empty errors when not in prod).

1591     Raises:
1592         ProductionConfigError: When ENV=prod and hard errors exist.
1593     """
1594     result = validate_settings(settings, strict=settings.is_prod)
1595     for warning in result.warnings:
1596         logger.warning("Config warning: %s", warning)
1597     if settings.is_prod and result.errors:
1598         for err in result.errors:
1599             logger.error("Config error: %s", err)
1600             raise ProductionConfigError(result.errors)
1601     if result.ok and settings.is_prod:
1602         logger.info("Production Settings validation passed")
1603     return result
1604 # === File: app/core/tenancy.py ===
1605 """Actor isolation + enterprise tenant filtering helpers."""
1606 from __future__ import annotations
1607 from sqlalchemy import Select, select
1608 from sqlalchemy.ext.asyncio import AsyncSession
1609 from sqlalchemy.orm import selectinload
1610 from app.config import settings
1611 from app.models.task import Task
1612 from app.models.test_suite import TestSuite
1613 from app.models.trace import Trace
1614 from app.utils.exceptions import NotFoundError
1615 def tenancy_enforced() -> bool:
1616     """Isolation is on when auth is enabled, or TENANCY_ENABLED is explicit."""
1617     return bool(settings.AUTH_ENABLED or settings.TENANCY_ENABLED)
1618 def admin_actors() -> set[str]:
1619     raw = settings.ADMIN_ACTORS or ""
1620     return {p.strip() for p in raw.split(",") if p.strip()}
1621 def is_admin(actor: str | None) -> bool:
1622     if not actor:
1623         return False
1624     return actor in admin_actors()
1625 def can_access_task(
1626     task: Task,
1627     actor: str | None,
1628     *,
1629     role: str | None = None,
1630 ) -> bool:
1631     """Return True if actor may read/mutate the task.

1633     Cross-owner access is granted to ADMIN/MANAGER/REVIEWER / enterprise
1634     system_admin / tenant_admin roles when RBAC is active; otherwise falls
1635     back to ADMIN_ACTORS + ownership checks.

1637     When multi-tenant is on and a tenant is selected, also enforce tenant_id.
1638     """
1639     try:
1640         from app.core.tenant_context import (
1641             current_tenant_id,
1642             multi_tenant_enabled,
1643         )
1644         if multi_tenant_enabled():

```

```

1657         tid = current_tenant_id()
1658         if tid:
1659             resource_tid = getattr(task, "tenant_id", None)
1660             if resource_tid and resource_tid != tid:
1661                 return False
1662             if not resource_tid:
1663                 return False
1664     except Exception:
1665         pass
1666     if not tenancy_enforced():
1667         return True
1668     if is_admin(actor):
1669         return True
1670     try:
1671         from app.core.rbac import CROSS_OWNER_ROLES, Role, rbac_enforced
1672         if rbac_enforced() and role:
1673             try:
1674                 r = Role.parse(str(role))
1675                 if r in CROSS_OWNER_ROLES:
1676                     return True
1677             except ValueError:
1678                 pass
1679     except Exception:
1680         pass
1681     owner = getattr(task, "created_by", None) or "anonymous"
1682     return owner == (actor or "anonymous")
1683 def ensure_task_access(
1684     task: Task | None,
1685     actor: str | None,
1686     task_id: str = "",
1687     *,
1688     role: str | None = None,
1689 ) -> Task:
1690     """Raise NotFoundError if missing or not accessible (no existence leak)."""
1691     if task is None or not can_access_task(task, actor, role=role):
1692         raise NotFoundError("Task", task_id or (task.id if task else ""))
1693     return task
1694 def apply_owner_filter(
1695     query: Select,
1696     actor: str | None,
1697     *,
1698     role: str | None = None,
1699 ) -> Select:
1700     """Restrict Task list queries to the current actor when tenancy is on.
1701
1702     Also applies ``tenant_id`` filter when multi-tenant context is active.
1703     """
1704     try:
1705         from app.core.tenant_context import apply_tenant_filter
1706         query = apply_tenant_filter(query, Task)
1707     except Exception:
1708         pass
1709     if not tenancy_enforced():
1710         return query
1711     if is_admin(actor):
1712         return query
1713     try:
1714         from app.core.rbac import CROSS_OWNER_ROLES, Role, rbac_enforced
1715         if rbac_enforced() and role:
1716             try:
1717                 if Role.parse(str(role)) in CROSS_OWNER_ROLES:
1718                     return query
1719             except ValueError:
1720                 pass
1721     except Exception:
1722         pass
1723     owner = actor or "anonymous"
1724     return query.where(Task.created_by == owner)

```



```

1729 def apply_trace_owner_filter(query: Select, actor: str | None) -> Select:
1730     """Restrict Trace queries to suites belonging to actor-owned tasks."""
1731     try:
1732         from app.core.tenant_context import apply_tenant_filter
1733         query = apply_tenant_filter(query, Trace)
1734     except Exception:
1735         pass
1736     if not tenancy_enforced():
1737         return query
1738     if is_admin(actor):
1739         return query
1740     owner = actor or "anonymous"
1741     return (
1742         query.join(TestSuite, Trace.test_suite_id == TestSuite.id)
1743         .join(Task, TestSuite.task_id == Task.id)
1744         .where(Task.created_by == owner)
1745     )
1746 async def load_task_for_suite(
1747     session: AsyncSession,
1748     suite_id: str,
1749     actor: str | None,
1750 ) -> Task:
1751     """Load parent Task for a test suite; enforce ownership."""
1752     result = await session.execute(
1753         select(Task)
1754         .join(TestSuite, TestSuite.task_id == Task.id)
1755         .where(TestSuite.id == suite_id)
1756     )
1757     task = result.scalar_one_or_none()
1758     if task is None or not can_access_task(task, actor):
1759         raise NotFoundError(" ", suite_id)
1760     return task
1761 async def load_trace_with_access(
1762     session: AsyncSession,
1763     trace_id: str,
1764     actor: str | None,
1765     *,
1766     with_scores: bool = True,
1767 ) -> Trace:
1768     """Load a Trace and ensure the caller owns the parent Task."""
1769     opts = [selectinload(Trace.metric_scores)] if with_scores else []
1770     result = await session.execute(
1771         select(Trace).options(*opts).where(Trace.id == trace_id)
1772     )
1773     trace = result.scalar_one_or_none()
1774     if trace is None:
1775         raise NotFoundError(" ", trace_id)
1776     suite_result = await session.execute(
1777         select(TestSuite).where(TestSuite.id == trace.test_suite_id)
1778     )
1779     suite = suite_result.scalar_one_or_none()
1780     if suite is None:
1781         raise NotFoundError(" ", trace_id)
1782     task_result = await session.execute(select(Task).where(Task.id == suite.task_id))
1783     task = task_result.scalar_one_or_none()
1784     if task is None or not can_access_task(task, actor):
1785         raise NotFoundError(" ", trace_id)
1786     return trace
1787 # === File: app/core/tenant_context.py ===
1788 """Request-scoped enterprise TenantContext (X-Tenant-ID)."""
1789 from __future__ import annotations
1790 from contextvars import ContextVar
1791 from dataclasses import dataclass
1792 from typing import Any
1793 from fastapi import Request
1794 from sqlalchemy import select
1795 from sqlalchemy.ext.asyncio import AsyncSession
1796 from app.config import settings
1797 from app.utils.exceptions import AgentFlowError, NotFoundError
1798 _tenant_ctx: ContextVar["TenantContext | None"] = ContextVar("tenant_ctx", default=None)

```

```

1801 class TenantAccessError(AgentFlowError):
1802     """Caller not allowed for the requested tenant."""
1803     def __init__(self, message: str = "Tenant access denied") -> None:
1804         super().__init__(message=message, status_code=403, detail=None)
1805 @dataclass(frozen=True)
1806 class TenantContext:
1807     """Resolved tenant for the current request."""
1808     tenant_id: str | None
1809     tenant_slug: str | None
1810     member_role: str | None
1811     enforced: bool
1812     @property
1813     def active(self) -> bool:
1814         return bool(self.tenant_id)
1815 def multi_tenant_enabled() -> bool:
1816     """Enterprise multi-tenant mode (org tables + X-Tenant-ID)."""
1817     return bool(getattr(settings, "MULTI_TENANT_ENABLED", False))
1818 def get_tenant_context() -> TenantContext | None:
1819     return _tenant_ctx.get()
1820 def set_tenant_context(ctx: TenantContext | None) -> None:
1821     _tenant_ctx.set(ctx)
1822 def current_tenant_id() -> str | None:
1823     ctx = get_tenant_context()
1824     return ctx.tenant_id if ctx else None
1825 def extract_tenant_header(request: Request) -> str | None:
1826     """Accept X-Tenant-ID (uuid or slug)."""
1827     raw = (
1828         request.headers.get("X-Tenant-ID")
1829         or request.headers.get("X-Tenant-Id")
1830         or request.headers.get("x-tenant-id")
1831         or ""
1832     ).strip()
1833     return raw or None
1834 async def resolve_tenant_context(
1835     session: AsyncSession,
1836     *,
1837     actor: str,
1838     header_value: str | None,
1839     system_role: str | None = None,
1840 ) -> TenantContext:
1841     """Resolve and validate tenant membership for ``actor``.
1843     - MULTI_TENANT off → empty context (legacy actor isolation only)
1844     - Header absent → empty tenant_id (optional scope)
1845     - Header present → must be valid tenant + membership (unless system_admin)
1846     """
1847     if not multi_tenant_enabled():
1848         ctx = TenantContext(
1849             tenant_id=None,
1850             tenant_slug=None,
1851             member_role=None,
1852             enforced=False,
1853         )
1854         set_tenant_context(ctx)
1855         return ctx
1856     if not header_value:
1857         ctx = TenantContext(
1858             tenant_id=None,
1859             tenant_slug=None,
1860             member_role=None,

```

```

1873         enforced=True,
1874     )
1875     set_tenant_context(ctx)
1876     return ctx
1877 from app.models.tenant import Tenant, TenantMember
1878 result = await session.execute(
1879     select(Tenant).where(
1880         (Tenant.id == header_value) | (Tenant.slug == header_value)
1881     )
1882 )
1883 tenant = result.scalar_one_or_none()
1884 if tenant is None or tenant.status == "deleted":
1885     raise NotFoundError("Tenant", header_value)
1886 if tenant.status == "suspended":
1887     raise TenantAccessError("Tenant is suspended")
1888 role_l = (system_role or "").lower()
1889 if role_l in {"system_admin", "admin"}:
1890     ctx = TenantContext(
1891         tenant_id=tenant.id,
1892         tenant_slug=tenant.slug,
1893         member_role="system_admin",
1894         enforced=True,
1895     )
1896     set_tenant_context(ctx)
1897     return ctx
1898 mem = await session.execute(
1899     select(TenantMember).where(
1900         TenantMember.tenant_id == tenant.id,
1901         TenantMember.user_id == actor,
1902     )
1903 )
1904 member = mem.scalar_one_or_none()
1905 if member is None:
1906     raise TenantAccessError(
1907         f"Actor {actor!r} is not a member of tenant {tenant.slug!r}"
1908     )
1909 ctx = TenantContext(
1910     tenant_id=tenant.id,
1911     tenant_slug=tenant.slug,
1912     member_role=member.role,
1913     enforced=True,
1914 )
1915 set_tenant_context(ctx)
1916 return ctx
1917 def apply_tenant_filter(query: Any, model: Any, tenant_id: str | None = None) -> Any:
1918     """Restrict query to a single tenant when multi-tenant is active and scoped.
1919
1920     Rows with ``tenant_id IS NULL`` are treated as legacy (visible only when
1921     filter is not enforced, or to system paths that opt out).
1922     """
1923     tid = tenant_id if tenant_id is not None else current_tenant_id()
1924     if not multi_tenant_enabled() or not tid:
1925         return query
1926     col = getattr(model, "tenant_id", None)
1927     if col is None:
1928         return query
1929     return query.where(col == tid)

```

```

1945 def ensure_tenant_resource(
1946     resource_tenant_id: str | None,
1947     *,
1948     resource: str = "Resource",
1949     resource_id: str = "",
1950 ) -> None:
1951     """Raise NotFound if resource belongs to another tenant (no existence leak)."""
1952     if not multi_tenant_enabled():
1953         return
1954     tid = current_tenant_id()
1955     if not tid:
1956         return
1957     if not resource_tenant_id or resource_tenant_id != tid:
1958         raise NotFoundError(resource, resource_id)
1959 def tenant_dict() -> dict[str, Any]:
1960     ctx = get_tenant_context()
1961     if not ctx:
1962         return {
1963             "multi_tenant_enabled": multi_tenant_enabled(),
1964             "tenant_id": None,
1965             "tenant_slug": None,
1966             "member_role": None,
1967         }
1968     return {
1969         "multi_tenant_enabled": multi_tenant_enabled(),
1970         "tenant_id": ctx.tenant_id,
1971         "tenant_slug": ctx.tenant_slug,
1972         "member_role": ctx.member_role,
1973         "enforced": ctx.enforced,
1974     }
1975 # === File: app/core/rbac.py ===
1976 """Role-Based Access Control (RBAC) for AgentFlow-Eval.

1978 Enterprise roles (v1.0)
1979 -----
1980 system_admin, tenant_admin, manager, reviewer, member, viewer

1982 Legacy aliases (still accepted)
1983 -----
1984 admin -> system_admin, user -> member, guest -> viewer

1986 Permissions
1987 -----
1988 task:* | evaluation:* | user:manage | system:config | audit:read
1989 tenant:create | tenant:manage | billing:read | billing:manage
1990 benchmark:create | benchmark:read

1992 API key format (`API_KEYS`)
1993 -----
1994 - ``secret``
1995 - ``secret:actor``
1996 - ``secret:actor:role`` e.g. ``sk-ops:alice:manager``

1998 Optional ``ACTOR_ROLES=alice:manager,bob:user`` overrides / fills role when not
1999 embedded in the key. Actors listed in ``ADMIN_ACTORS`` default to SYSTEM_ADMIN.
2000 """
2001 from __future__ import annotations
2002 import functools
2003 import inspect
2004 import logging
2005 from collections.abc import Callable, Collection
2006 from enum import Enum
2007 from typing import Any, ParamSpec, TypeVar
2008 from fastapi import Depends, Request
2009 from app.config import settings
2010 from app.utils.exceptions import AgentFlowError
2011 logger = logging.getLogger(__name__)
2012 P = ParamSpec("P")
2013 R = TypeVar("R")

```

```

2017 class Role(str, Enum):
2018     """Roles ordered from most to least privileged.

2020     Enterprise (canonical): system_admin, tenant_admin, manager, reviewer,
2021     member, viewer.

2023     Legacy members (admin/user/guest) remain for backward-compatible API keys
2024     and tests; they share permission sets with their enterprise equivalents.
2025     """
2026     SYSTEM_ADMIN = "system_admin"
2027     TENANT_ADMIN = "tenant_admin"
2028     MANAGER = "manager"
2029     REVIEWER = "reviewer"
2030     MEMBER = "member"
2031     VIEWER = "viewer"
2032     ADMIN = "admin"
2033     USER = "user"
2034     GUEST = "guest"
2035     @classmethod
2036     def parse(cls, value: str | None, default: "Role | None" = None) -> "Role":
2037         """Parse a role string (case-insensitive); maps legacy names."""
2038         if value is None or not str(value).strip():
2039             if default is not None:
2040                 return default
2041             raise ValueError("empty role")
2042         key = str(value).strip().lower().replace("-", "_")
2043         aliases = {
2044             "sysadmin": cls.SYSTEM_ADMIN,
2045             "systemadmin": cls.SYSTEM_ADMIN,
2046             "owner": cls.TENANT_ADMIN,
2047             "org_admin": cls.TENANT_ADMIN,
2048         }
2049         if key in aliases:
2050             return aliases[key]
2051         for role in cls:
2052             if role.value == key or role.name.lower() == key:
2053                 return role
2054         raise ValueError(f"unknown role: {value!r}")
2055     def canonical(self) -> "Role":
2056         """Map legacy roles to enterprise names."""
2057         if self is Role.ADMIN:
2058             return Role.SYSTEM_ADMIN
2059         if self is Role.USER:
2060             return Role.MEMBER
2061         if self is Role.GUEST:
2062             return Role.VIEWER
2063         return self
2064 class Permission(str, Enum):
2065     """Fine-grained permissions (resource:action)."""
2066     TASK_CREATE = "task:create"
2067     TASK_READ = "task:read"
2068     TASK_UPDATE = "task:update"
2069     TASK_DELETE = "task:delete"
2070     TASK_EXECUTE = "task:execute"
2071     TASK_CANCEL = "task:cancel"
2072     EVALUATION_READ = "evaluation:read"
2073     EVALUATION_SUBMIT = "evaluation:submit"
2074     EVALUATION_APPROVE = "evaluation:approve"
2075     USER_MANAGE = "user:manage"
2076     SYSTEM_CONFIG = "system:config"

```

```

2089     AUDIT_READ = "audit:read"
2090     TENANT_CREATE = "tenant:create"
2091     TENANT_MANAGE = "tenant:manage"
2092     BILLING_READ = "billing:read"
2093     BILLING_MANAGE = "billing:manage"
2094     BENCHMARK_CREATE = "benchmark:create"
2095     BENCHMARK_READ = "benchmark:read"
2096     _ALL_TASK = frozenset(
2097     {
2098         Permission.TASK_CREATE,
2099         Permission.TASK_READ,
2100         Permission.TASK_UPDATE,
2101         Permission.TASK_DELETE,
2102         Permission.TASK_EXECUTE,
2103         Permission.TASK_CANCEL,
2104     }
2105 )
2106     _ALL_EVAL = frozenset(
2107     {
2108         Permission.EVALUATION_READ,
2109         Permission.EVALUATION_SUBMIT,
2110         Permission.EVALUATION_APPROVE,
2111     }
2112 )
2113     _TENANT_OPS = frozenset({Permission.TENANT_MANAGE})
2114     _BILLING = frozenset({Permission.BILLING_READ, Permission.BILLING_MANAGE})
2115     _BENCHMARK = frozenset({Permission.BENCHMARK_CREATE, Permission.BENCHMARK_READ})
2116     _SYSTEM_ADMIN_PERMS = frozenset(Permission)
2117     _TENANT_ADMIN_PERMS = frozenset(
2118     {
2119         *_ALL_TASK,
2120         *_ALL_EVAL,
2121         Permission.AUDIT_READ,
2122         Permission.USER_MANAGE,
2123         *_TENANT_OPS,
2124         *_BILLING,
2125         *_BENCHMARK,
2126     }
2127 )
2128     _MANAGER_PERMS = frozenset(
2129     {
2130         *_ALL_TASK,
2131         *_ALL_EVAL,
2132         Permission.AUDIT_READ,
2133         Permission.BILLING_READ,
2134         Permission.BENCHMARK_CREATE,
2135         Permission.BENCHMARK_READ,
2136     }
2137 )
2138     _REVIEWER_PERMS = frozenset(
2139     {
2140         Permission.TASK_READ,
2141         Permission.EVALUATION_READ,
2142         Permission.EVALUATION_SUBMIT,
2143         Permission.EVALUATION_APPROVE,

```

```

2161         Permission.AUDIT_READ,
2162         Permission.BENCHMARK_READ,
2163     }
2164 )
2165 _MEMBER_PERMS = frozenset(
2166     {
2167         *_ALL_TASK,
2168         Permission.EVALUATION_READ,
2169         Permission.EVALUATION_SUBMIT,
2170         Permission.BENCHMARK_READ,
2171         Permission.BILLING_READ,
2172     }
2173 )
2174 _VIEWER_PERMS = frozenset(
2175     {
2176         Permission.TASK_READ,
2177         Permission.EVALUATION_READ,
2178         Permission.BENCHMARK_READ,
2179     }
2180 )
2181 ROLE_PERMISSIONS: dict[Role, frozenset[Permission]] = {
2182     Role.SYSTEM_ADMIN: _SYSTEM_ADMIN_PERMS,
2183     Role.TENANT_ADMIN: _TENANT_ADMIN_PERMS,
2184     Role.MANAGER: _MANAGER_PERMS,
2185     Role.REVIEWER: _REVIEWER_PERMS,
2186     Role.MEMBER: _MEMBER_PERMS,
2187     Role.VIEWER: _VIEWER_PERMS,
2188     Role.ADMIN: _SYSTEM_ADMIN_PERMS,
2189     Role.USER: _MEMBER_PERMS,
2190     Role.GUEST: _VIEWER_PERMS,
2191 }
2192 CROSS_OWNER_ROLES: frozenset[Role] = frozenset(
2193     {
2194         Role.SYSTEM_ADMIN,
2195         Role.TENANT_ADMIN,
2196         Role.MANAGER,
2197         Role.REVIEWER,
2198         Role.ADMIN,
2199     }
2200 )
2201 CROSS_TENANT_ROLES = CROSS_OWNER_ROLES
2202 class ForbiddenError(AgentFlowError):
2203     """Permission denied (HTTP 403)."""
2204     def __init__(
2205         self,
2206         message: str = "Forbidden",
2207         detail: Any = None,
2208     ) -> None:
2209         super().__init__(message=message, status_code=403, detail=detail)
2210 def rbac_enforced() -> bool:
2211     """Return True when RBAC permission checks must be applied.
2212
2213     Requires both AUTH and RBAC flags so local Eager mode stays frictionless.
2214     """
2215     return bool(
2216         getattr(settings, "AUTH_ENABLED", False)
2217         and getattr(settings, "RBAC_ENABLED", True)
2218     )

```

```

2233 def default_role() -> Role:
2234     """Default role for authenticated actors without an explicit mapping."""
2235     raw = getattr(settings, "DEFAULT_ROLE", None) or "member"
2236     try:
2237         return Role.parse(str(raw), default=Role.MEMBER)
2238     except ValueError:
2239         return Role.MEMBER
2240 def parse_actor_roles(raw: str | None = None) -> dict[str, Role]:
2241     """Parse ``ACTOR_ROLES`` config: ``alice:manager,bob:member``."""
2242     text = raw if raw is not None else getattr(settings, "ACTOR_ROLES", "") or ""
2243     mapping: dict[str, Role] = {}
2244     for part in str(text).split(","):
2245         part = part.strip()
2246         if not part or ":" not in part:
2247             continue
2248         actor, role_s = part.split(":", 1)
2249         actor = actor.strip()
2250         try:
2251             mapping[actor] = Role.parse(role_s.strip())
2252         except ValueError:
2253             logger.warning("Ignoring invalid ACTOR_ROLES entry: %s", part)
2254     return mapping
2255 def resolve_role_for_actor(actor: str, explicit: Role | None = None) -> Role:
2256     """Resolve role for an actor name.
2257
2258     Priority: explicit (from API key) → ACTOR_ROLES → ADMIN_ACTORS → DEFAULT_ROLE.
2259     """
2260     if explicit is not None:
2261         return explicit
2262     from app.core.tenancy import admin_actors
2263     if actor in admin_actors():
2264         return Role.SYSTEM_ADMIN
2265     mapped = parse_actor_roles().get(actor)
2266     if mapped is not None:
2267         return mapped
2268     return default_role()
2269 def permissions_for(role: Role) -> frozenset[Permission]:
2270     """Return the permission set granted to ``role``."""
2271     return ROLE_PERMISSIONS.get(role, frozenset())
2272 def has_permission(
2273     role: Role | str | None,
2274     permission: Permission | str,
2275 ) -> bool:
2276     """Check whether ``role`` includes ``permission``."""
2277     if role is None:
2278         return False
2279     try:
2280         r = role if isinstance(role, Role) else Role.parse(str(role))
2281     except ValueError:
2282         return False
2283     try:
2284         p = (
2285             permission
2286             if isinstance(permission, Permission)
2287             else Permission(str(permission))
2288         )
2289     except ValueError:
2290         return False
2291     return p in permissions_for(r)

```



```

2305 def has_any_permission(
2306     role: Role | str | None,
2307     permissions: Collection[Permission | str],
2308 ) -> bool:
2309     """True if role has at least one of the given permissions."""
2310     return any(has_permission(role, p) for p in permissions)
2311 def has_all_permissions(
2312     role: Role | str | None,
2313     permissions: Collection[Permission | str],
2314 ) -> bool:
2315     """True if role has every given permission."""
2316     return all(has_permission(role, p) for p in permissions)
2317 def get_request_role(request: Request) -> Role:
2318     """Resolve the caller's role from request state / identity."""
2319     auth = getattr(request.state, "auth", None)
2320     if auth is not None and getattr(auth, "role", None) is not None:
2321         role = auth.role
2322         if isinstance(role, Role):
2323             return role
2324         try:
2325             return Role.parse(str(role))
2326         except ValueError:
2327             pass
2328     actor = getattr(request.state, "actor", None) or "anonymous"
2329     if actor in {"public", "anonymous"} and not rbac_enforced():
2330         return Role.SYSTEM_ADMIN
2331     if actor == "public":
2332         return Role.VIEWER
2333     return resolve_role_for_actor(actor)
2334 def ensure_permission(
2335     role: Role | str | None,
2336     permission: Permission | str,
2337     *,
2338     message: str | None = None,
2339 ) -> None:
2340     """Raise :class:`ForbiddenError` if permission is missing (when RBAC on)."""
2341     if not rbac_enforced():
2342         return
2343     if has_permission(role, permission):
2344         return
2345     perm_s = permission.value if isinstance(permission, Permission) else str(permission)
2346     role_s = role.value if isinstance(role, Role) else str(role or "none")
2347     raise ForbiddenError(
2348         message or f"Missing permission: {perm_s}",
2349         detail={"required": perm_s, "role": role_s},
2350     )
2351 def ensure_resource_access(
2352     *,
2353     role: Role | str | None,
2354     actor: str | None,
2355     owner: str | None,
2356     permission: Permission | str,
2357     resource: str = "Resource",
2358     resource_id: str = "",
2359     hide_existence: bool = True,

```

```

2377 ) -> None:
2378     """Permission + ownership check for a single resource.

2380     SYSTEM_ADMIN / TENANT_ADMIN / MANAGER / REVIEWER may cross *owners*
2381     within the same tenant for permitted actions.
2382     MEMBER / VIEWER are limited to resources they own (`owner == actor`).

2384     Args:
2385         role: Caller role.
2386         actor: Caller actor name.
2387         owner: Resource owner (e.g. task.created_by).
2388         permission: Required permission.
2389         resource: Resource type label for errors.
2390         resource_id: Resource id for errors.
2391         hide_existence: If True, deny with 404 instead of 403 for foreign rows.
2392
2393     ensure_permission(role, permission)
2394     if not rbac_enforced():
2395         return
2396     try:
2397         r = role if isinstance(role, Role) else Role.parse(str(role or "viewer"))
2398     except ValueError:
2399         r = Role.VIEWER
2400     if r in CROSS_OWNER_ROLES:
2401         return
2402     owner_s = (owner or "anonymous").strip() or "anonymous"
2403     actor_s = (actor or "anonymous").strip() or "anonymous"
2404     if owner_s == actor_s:
2405         return
2406     if hide_existence:
2407         from app.utils.exceptions import NotFoundError
2408         raise NotFoundError(resource, resource_id)
2409     raise ForbiddenError(
2410         f"Not allowed to access {resource}",
2411         detail={"resource_id": resource_id, "owner": owner_s},
2412     )
2413 def _extract_request(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Request | None:
2414     if "request" in kwargs and isinstance(kwargs["request"], Request):
2415         return kwargs["request"]
2416     for a in args:
2417         if isinstance(a, Request):
2418             return a
2419     return None
2420 def _check_permissions_on_request(
2421     request: Request | None,
2422     permissions: tuple[Permission | str, ...],
2423     *,
2424     require_all: bool,
2425 ) -> None:
2426     if request is None:
2427         if rbac_enforced():
2428             raise ForbiddenError(
2429                 "Permission check requires Request in endpoint signature"
2430             )
2431         return
2432     role = get_request_role(request)
2433     if require_all:
2434         for perm in permissions:
2435             ensure_permission(role, perm)
2436         return
2437     if rbac_enforced() and not has_any_permission(role, permissions):
2438         raise ForbiddenError(
2439             "Missing one of required permissions",
2440             detail={
2441                 "required_any": [
2442                     p.value if isinstance(p, Permission) else str(p)
2443                     for p in permissions
2444                 ],
2445                 "role": role.value,
2446             },
2447         )

```

```

2449 def _preserve_fastapi_signature(
2450     wrapper: Callable[..., Any], fn: Callable[..., Any]
2451 ) -> None:
2452     """Copy resolved type hints onto the wrapper so FastAPI can introspect it.

2454     With ``from __future__ import annotations``, string annotations must be
2455     resolved against the *original* function globals. FastAPI reads
2456     ``inspect.signature`` annotations, so we rebuild the signature with
2457     concrete types (not ForwardRef strings).
2458     """
2459     from typing import get_type_hints
2460     sig = inspect.signature(fn)
2461     try:
2462         hints = get_type_hints(fn)
2463     except Exception:
2464         hints = {}
2465     new_params: list[inspect.Parameter] = []
2466     for name, param in sig.parameters.items():
2467         ann = hints.get(name, param.annotation)
2468         new_params.append(param.replace(annotation=ann))
2469     ret = hints.get("return", sig.return_annotation)
2470     wrapper.__signature__ = sig.replace(
2471         parameters=new_params,
2472         return_annotation=ret,
2473     )
2474     ann_map: dict[str, Any] = {p.name: p.annotation for p in new_params}
2475     if ret is not inspect.Signature.empty:
2476         ann_map["return"] = ret
2477     wrapper.__annotations__ = ann_map
2478 def require_permission(
2479     *permissions: Permission | str,
2480     require_all: bool = True,
2481 ) -> Callable[[Callable[P, R]], Callable[P, R]]:
2482     """Decorator enforcing one or more permissions.

2484     Works for both ``async def`` and plain ``def`` callables that receive a
2485     Starlette/FastAPI ``Request`` (as argument or ``request=`` kwarg).

2487     Args:
2488         permissions: Required permission(s).
2489         require_all: If True, all must be present; else any one is enough.
2490     """
2491     if not permissions:
2492         raise ValueError("require_permission() needs at least one permission")
2493     def decorator(fn: Callable[P, R]) -> Callable[P, R]:
2494         if inspect.iscoroutinefunction(fn):
2495             @functools.wraps(fn)
2496             async def async_wrapper(*args: P.args, **kwargs: P.kwargs) -> Any:
2497                 request = _extract_request(args, kwargs)
2498                 _check_permissions_on_request(
2499                     request, permissions, require_all=require_all
2500                 )
2501                 return await fn(*args, **kwargs)
2502             _preserve_fastapi_signature(async_wrapper, fn)
2503             return async_wrapper
2504         @functools.wraps(fn)

```

```

2521     def sync_wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
2522         request = _extract_request(args, kwargs)
2523         _check_permissions_on_request(request, permissions, require_all=require_all)
2524         return fn(*args, **kwargs)
2525         _preserve_fastapi_signature(sync_wrapper, fn)
2526     return sync_wrapper
2527     return decorator
2528 def RequirePermission(
2529     *permissions: Permission | str,
2530     require_all: bool = True,
2531 ) -> Any:
2532     """FastAPI dependency that enforces permissions and returns the role.

2534     Usage::

2536         @router.post("")
2537         async def create_task(
2538             request: Request,
2539             _role: Role = RequirePermission(Permission.TASK_CREATE),
2540         ):
2541             ...
2542     """
2543     async def _dependency(request: Request) -> Role:
2544         role = get_request_role(request)
2545         if require_all:
2546             for perm in permissions:
2547                 ensure_permission(role, perm)
2548         else:
2549             if rbac_enforced() and not has_any_permission(role, permissions):
2550                 raise ForbiddenError(
2551                     "Missing one of required permissions",
2552                     detail={
2553                         "required_any": [
2554                             p.value if isinstance(p, Permission) else str(p)
2555                             for p in permissions
2556                         ],
2557                         "role": role.value,
2558                     },
2559                 )
2560             request.state.role = role
2561             return role
2562     return Depends(_dependency)
2563 def matrix_as_dict() -> dict[str, list[str]]:
2564     """Export role->permission matrix for docs / settings API."""
2565     return {
2566         role.value: sorted(p.value for p in perms)
2567         for role, perms in ROLE_PERMISSIONS.items()
2568     }
2569 # === File: app/core/seed.py ===
2570 """Seed data for AgentFlow-Eval demonstration (soft-copyright / open-source demo).

2572 Populates:
2573 - Demo task + suites + traces (incl. failure for Diagnosis)
2574 - Dimension metric scores (scorecard-aligned 40/40/20)
2575 - AOLS agent_logs for Dashboard / Monitoring

```

```

2593 - Multi-variant Experiment for /experiments compare UI
2595 Run: ``python -m app.core.seed`` or ``python -m app.core.seed --force``
2596 """
2597 from __future__ import annotations
2598 from datetime import datetime, timedelta, timezone
2599 from uuid import uuid4
2600 SAMPLE_SCORECARD = {
2601     "version": 1,
2602     "name": "default_agent_eval",
2603     "llm_refine": True,
2604     "dimensions": [
2605         {
2606             "key": "tool_accuracy",
2607             "label": " ",
2608             "weight": 40,
2609             "description": " ",
2610             "method": "rule_tool",
2611         },
2612         {
2613             "key": "answer_correctness",
2614             "label": " ",
2615             "weight": 40,
2616             "description": " expected_output ",
2617             "method": "llm_or_lexical",
2618         },
2619         {
2620             "key": "reasoning_coherence",
2621             "label": " ",
2622             "weight": 20,
2623             "description": " ",
2624             "method": "llm_only",
2625         },
2626     ],
2627 }
2628 SAMPLE_AGENT_CONFIG = {
2629     "runner": "openai",
2630     "model": "gpt-4o-mini",
2631     "temperature": 0,
2632     "max_iterations": 5,
2633     "max_tokens": 4096,
2634     "scorecard": SAMPLE_SCORECARD,
2635 }
2636 SAMPLE_AGENT_CONFIG_CANDIDATE = {
2637     "runner": "openai",
2638     "model": "gpt-4o",
2639     "temperature": 0,
2640     "max_iterations": 5,
2641     "max_tokens": 4096,
2642     "scorecard": SAMPLE_SCORECARD,
2643 }
2644 SAMPLE_TASK = {
2645     "name": " Agent (Demo)",
2646     "description": (
2647         " :
2648     );"

```

```

2665         " / Trace AOLS "
2666         "Seed , /experiments ."
2667     ),
2668     "agent_config": SAMPLE_AGENT_CONFIG,
2669 }
2670 DEMO_EXPERIMENT_NAME = "Demo (mini vs 4o)"
2671 SAMPLE_TEST_SUITES = [
2672     {
2673         "user_query": " ?",
2674         "expected_output": " , 25 .",
2675         "expected_tools": ["get_weather"],
2676         "outcome": "success",
2677     },
2678     {
2679         "user_query": " 15 x 37 ?",
2680         "expected_output": "555",
2681         "expected_tools": ["calculator"],
2682         "outcome": "success",
2683     },
2684     {
2685         "user_query": " ",
2686         "expected_output": " ",
2687         "expected_tools": ["flight_search", "book_flight"],
2688         "outcome": "success",
2689     },
2690     {
2691         "user_query": " zhangsan@company.com , Meeting, Hello.",
2692         "expected_output": " zhangsan@company.com ",
2693         "expected_tools": ["send_email"],
2694         "outcome": "success",
2695     },
2696     {
2697         "user_query": " ?",
2698         "expected_output": " ",
2699         "expected_tools": [],
2700         "outcome": "success",
2701     },
2702     {
2703         "user_query": " ( )",
2704         "expected_output": " ",
2705         "expected_tools": ["flight_search"],
2706         "outcome": "failed",
2707     },
2708 ]
2709 def _success_steps(query: str, tools: list[str]) -> list[dict]:
2710     steps: list[dict] = [
2711         {
2712             "type": "thought",
2713             "content": f" :{query[:80]}. .",
2714             "tokens": 40,
2715         },
2716     ]
2717     for tool in tools or ["final_answer"]:
2718         steps.append(
2719             {

```

```

2737         "type": "action",
2738         "tool_name": tool if tool != "final_answer" else "",
2739         "action": tool,
2740         "tool_input": query[:60],
2741         "content": f"call {tool}",
2742         "tokens": 25,
2743     }
2744 )
2745 steps.append(
2746     {
2747         "type": "observation",
2748         "content": f"{tool} ok: demo observation for seed",
2749         "tokens": 30,
2750     }
2751 )
2752 steps.append(
2753     {
2754         "type": "final_answer",
2755         "content": "        (seed)",
2756         "tokens": 20,
2757     }
2758 )
2759 return steps
2760 def _failed_loop_steps(query: str) -> list[dict]:
2761     """Repeated tool calls + error observations → diagnosis agent_loop / tool_failure."""
2762     steps: list[dict] = [
2763         {"type": "thought", "content": f"    :{query[:60]}", "tokens": 50},
2764     ]
2765     for i in range(4):
2766         steps.append(
2767             {
2768                 "type": "action",
2769                 "tool_name": "flight_search",
2770                 "action": "flight_search",
2771                 "tool_input": "PEK-SHA next monday",
2772                 "content": "retry flight_search",
2773                 "tokens": 35 + i,
2774             }
2775         )
2776         steps.append(
2777             {
2778                 "type": "observation",
2779                 "content": "error: timeout connecting to flight API (rate limit 429)",
2780                 "tokens": 20,
2781             }
2782         )
2783     steps.append(
2784         {
2785             "type": "final_answer",
2786             "content": "        ,        ",
2787             "tokens": 15,
2788         }
2789     )
2790     return steps

```

```

2809 async def seed_database(async_session_factory, *, force: bool = False) -> None:
2810     """Insert seed data into the database.

2812     Args:
2813         async_session_factory: SQLAlchemy async session factory.
2814         force: If True, insert demo task even when other tasks already exist
2815             (skips only when a demo-named task is present).
2816     """
2817     from sqlalchemy import select, func
2818     from app.models.agent_log import AgentLog
2819     from app.models.metric_score import MetricScore
2820     from app.models.task import Task, TaskStatus
2821     from app.models.test_suite import TestSuite
2822     from app.models.trace import Trace, TraceStatus
2823     async with async_session_factory() as session:
2824         if not force:
2825             result = await session.execute(select(func.count(Task.id)))
2826             count = result.scalar() or 0
2827             if count > 0:
2828                 return
2829         else:
2830             existing = await session.execute(
2831                 select(Task).where(Task.name == SAMPLE_TASK["name"]).limit(1)
2832             )
2833             existing_task = existing.scalar_one_or_none()
2834             if existing_task is not None:
2835                 exp_id, labels = await _seed_experiment_pair(
2836                     session,
2837                     base_task_id=existing_task.id,
2838                     now=datetime.now(timezone.utc),
2839                 )
2840                 await session.commit()
2841                 if exp_id:
2842                     return
2843             now = datetime.now(timezone.utc)
2844             task_id = str(uuid4())
2845             task = Task(
2846                 id=task_id,
2847                 name=SAMPLE_TASK["name"],
2848                 description=SAMPLE_TASK["description"],
2849                 agent_config=SAMPLE_TASK["agent_config"],
2850                 status=TaskStatus.COMPLETED,
2851                 created_by="demo",
2852             )
2853             session.add(task)
2854             await session.flush()
2855             suite_ids: list[str] = []
2856             trace_ids: list[str] = []
2857             log_rows = 0
2858             for idx, suite_data in enumerate(SAMPLE_TEST_SUITES):
2859                 suite_id = str(uuid4())
2860                 suite = TestSuite(
2861                     id=suite_id,
2862                     task_id=task.id,
2863                     user_query=suite_data["user_query"],

```



```

2881         expected_output=suite_data["expected_output"],
2882         expected_tools=suite_data["expected_tools"],
2883     )
2884     session.add(suite)
2885     suite_ids.append(suite_id)
2886     await session.flush()
2887     failed = suite_data.get("outcome") == "failed"
2888     steps = (
2889         _failed_loop_steps(suite_data["user_query"])
2890         if failed
2891         else _success_steps(
2892             suite_data["user_query"],
2893             list(suite_data.get("expected_tools") or []),
2894         )
2895     )
2896     tokens = sum(int(s.get("tokens") or 0) for s in steps)
2897     created = now - timedelta(
2898         days=(len(SAMPLE_TEST_SUITES) - idx) % 6, hours=idx
2899     )
2900     trace_id = str(uuid4())
2901     tr = Trace(
2902         id=trace_id,
2903         test_suite_id=suite_id,
2904         user_query=suite_data["user_query"],
2905         steps=steps,
2906         total_tokens=tokens,
2907         prompt_tokens=int(tokens * 0.6),
2908         completion_tokens=int(tokens * 0.4),
2909         response_time_ms=800 + idx * 120 + (2000 if failed else 0),
2910         status=TraceStatus.FAILED if failed else TraceStatus.SUCCESS,
2911         cost=round(tokens * 0.000002, 6),
2912         agent_version="demo-seed-1.0",
2913         model_version="gpt-4o-mini",
2914     )
2915     if hasattr(tr, "created_at"):
2916         try:
2917             tr.created_at = created
2918         except Exception:
2919             pass
2920     session.add(tr)
2921     trace_ids.append(trace_id)
2922     await session.flush()
2923     for metric_name, score, reason in _dimension_scores(
2924         failed=failed, idx=idx, variant="baseline"
2925     ):
2926         session.add(
2927             MetricScore(
2928                 id=str(uuid4()),
2929                 trace_id=trace_id,
2930                 metric_name=metric_name,
2931                 score=score,
2932                 reason=reason,
2933                 confidence=0.75 if not failed else 0.55,
2934             )
2935         )

```

```

2953     events = [
2954         ("info", "evaluation.running", {"suite_index": idx}),
2955         ("info", "agent.started", {"model": "gpt-4o-mini"}),
2956         (
2957             "info" if not failed else "error",
2958             "agent.completed" if not failed else "agent.failed",
2959             {"tokens": tokens, "latency_ms": tr.response_time_ms},
2960         ),
2961         (
2962             "info",
2963             "llm.request",
2964             {"tokens": tokens, "latency_ms": 200 + idx * 10},
2965         ),
2966     ]
2967     if failed:
2968         events.append(
2969             (
2970                 "error",
2971                 "tool.failed",
2972                 {"tool": "flight_search", "error": "timeout 429"},
2973             )
2974         )
2975     else:
2976         events.append(
2977             (
2978                 "info",
2979                 "tool.completed",
2980                 {"tool": (suite_data.get("expected_tools") or ["none"])[0]},
2981             )
2982         )
2983     for level, event, payload in events:
2984         session.add(
2985             AgentLog(
2986                 id=str(uuid4()),
2987                 level=level,
2988                 event=event,
2989                 service="agentflow-api",
2990                 trace_id=trace_id[:32],
2991                 task_id=task_id,
2992                 payload={
2993                     "seed": True,
2994                     "total_tokens": tokens,
2995                     "latency_ms": tr.response_time_ms,
2996                     **payload,
2997                 },
2998             )
2999         )
3000     log_rows += 1
3001     for d in range(7):
3002         for k in range(3):
3003             session.add(
3004                 AgentLog(
3005                     id=str(uuid4()),
3006                     level="info" if k % 3 else "warning",
3007                     event="http.request" if k else "evaluation.suite_complete",

```

```

3025         service="agentflow-api",
3026         trace_id=f"seed-day-{d}",
3027         task_id=task_id,
3028         payload={
3029             "seed": True,
3030             "total_tokens": 100 + d * 10 + k,
3031             "latency_ms": 50 + k * 15,
3032             "day_offset": d,
3033         },
3034     )
3035     )
3036     log_rows += 1
3037     exp_id, run_labels = await _seed_experiment_pair(
3038         session,
3039         base_task_id=task_id,
3040         now=now,
3041     )
3042     await session.commit()
3043     if exp_id:
3044         print(
3045             "Next: /dashboard · /experiments · demo task Trace + ScoreCard · /diagnosis"
3046         )
3047 def _dimension_scores(
3048     *,
3049     failed: bool,
3050     idx: int,
3051     variant: str = "baseline",
3052 ) -> list[tuple[str, float, str]]:
3053     """Scorecard-aligned seed scores (tool/answer/coherence)."""
3054     boost = 3.0 if variant == "candidate" else 0.0
3055     if failed:
3056         return [
3057             ("tool_accuracy", 12.0, "seed: tool loop / timeout"),
3058             ("answer_correctness", 15.0, "seed: incomplete answer"),
3059             ("reasoning_coherence", 8.0, "seed: repeated steps"),
3060         ]
3061     tool = min(40.0, max(0.0, 36.0 - idx * 0.8 + boost))
3062     ans = min(40.0, max(0.0, 34.0 - idx * 0.5 + boost))
3063     coh = min(20.0, max(0.0, 17.5 - idx * 0.3 + boost * 0.5))
3064     return [
3065         ("tool_accuracy", round(tool, 1), "seed demo"),
3066         ("answer_correctness", round(ans, 1), "seed demo"),
3067         ("reasoning_coherence", round(coh, 1), "seed demo"),
3068     ]
3069 async def _seed_variant_task(
3070     session,
3071     *,
3072     label: str,
3073     agent_config: dict,
3074     snapshot: list[dict],
3075     variant: str,
3076     model_version: str,
3077 ) -> str:
3078     """Materialize one completed variant task with suites/traces/metrics."""
3079     from app.models.metric_score import MetricScore

```

```

3097 from app.models.task import Task, TaskStatus
3098 from app.models.test_suite import TestSuite
3099 from app.models.trace import Trace, TraceStatus
3100 task_id = str(uuid4())
3101 session.add(
3102     Task(
3103         id=task_id,
3104         name=f"Demo . {label}",
3105         description=f"Seed {label}.",
3106         agent_config=agent_config,
3107         status=TaskStatus.COMPLETED,
3108         created_by="demo",
3109     )
3110 )
3111 await session.flush()
3112 for idx, case in enumerate(snapshot):
3113     suite_id = str(uuid4())
3114     session.add(
3115         TestSuite(
3116             id=suite_id,
3117             task_id=task_id,
3118             user_query=case["user_query"],
3119             expected_output=case["expected_output"],
3120             expected_tools=case["expected_tools"],
3121         )
3122     )
3123     await session.flush()
3124     steps = _success_steps(
3125         case["user_query"], list(case.get("expected_tools") or [])
3126     )
3127     tokens = sum(int(s.get("tokens") or 0) for s in steps)
3128     token_mul = 1.15 if variant == "candidate" else 1.0
3129     tokens = int(tokens * token_mul)
3130     tid = str(uuid4())
3131     session.add(
3132         Trace(
3133             id=tid,
3134             test_suite_id=suite_id,
3135             user_query=case["user_query"],
3136             steps=steps,
3137             total_tokens=tokens,
3138             prompt_tokens=int(tokens * 0.6),
3139             completion_tokens=int(tokens * 0.4),
3140             response_time_ms=int(
3141                 (750 + idx * 90) * (1.1 if variant == "candidate" else 1.0)
3142             ),
3143             status=TraceStatus.SUCCESS,
3144             cost=round(tokens * 0.0000025, 6),
3145             agent_version="demo-seed-1.0",
3146             model_version=model_version,
3147         )
3148     )
3149     await session.flush()
3150     for metric_name, score, reason in _dimension_scores(
3151         failed=False, idx=idx, variant=variant
3152     ):
3153         session.add(
3154             MetricScore(
3155                 id=str(uuid4()),
3156                 trace_id=tid,
3157                 metric_name=metric_name,
3158                 score=score,
3159                 reason=reason,
3160                 confidence=0.8,
3161             )
3162         )
3163 return task_id

```

```

3169 async def _seed_experiment_pair(session, *, base_task_id: str, now: datetime):
3170     """Create two fair variant tasks + Experiment for /experiments compare."""
3171     from app.models.experiment import Experiment, ExperimentRun
3172     from sqlalchemy import select
3173     _ = now
3174     existing = await session.execute(
3175         select(Experiment).where(Experiment.name == DEMO_EXPERIMENT_NAME).limit(1)
3176     )
3177     if existing.scalar_one_or_none() is not None:
3178         return None, []
3179     snapshot = [
3180         {
3181             "user_query": s["user_query"],
3182             "expected_output": s["expected_output"],
3183             "expected_tools": list(s.get("expected_tools") or []),
3184         }
3185         for s in SAMPLE_TEST_SUITES
3186         if s.get("outcome") != "failed"
3187     ][0:3]
3188     baseline_id = await _seed_variant_task(
3189         session,
3190         label="gpt-4o-mini",
3191         agent_config=SAMPLE_AGENT_CONFIG,
3192         snapshot=snapshot,
3193         variant="baseline",
3194         model_version="gpt-4o-mini",
3195     )
3196     candidate_id = await _seed_variant_task(
3197         session,
3198         label="gpt-4o",
3199         agent_config=SAMPLE_AGENT_CONFIG_CANDIDATE,
3200         snapshot=snapshot,
3201         variant="candidate",
3202         model_version="gpt-4o",
3203     )
3204     exp_id = str(uuid4())
3205     session.add(
3206         Experiment(
3207             id=exp_id,
3208             name=DEMO_EXPERIMENT_NAME,
3209             description=(
3210                 "Seed      :      3      ,      gpt-4o-mini      gpt-4o."
3211                 "Best /      / Token /      "
3212             ),
3213             base_task_id=base_task_id,
3214             suite_snapshot=snapshot,
3215             created_by="demo",
3216         )
3217     )
3218     await session.flush()
3219     session.add(
3220         ExperimentRun(
3221             id=str(uuid4()),
3222             experiment_id=exp_id,
3223             task_id=baseline_id,
3224             label="gpt-4o-mini",
3225             agent_config=SAMPLE_AGENT_CONFIG,
3226         )
3227     )
3228     session.add(
3229         ExperimentRun(
3230             id=str(uuid4()),
3231             experiment_id=exp_id,
3232             task_id=candidate_id,
3233             label="gpt-4o",
3234             agent_config=SAMPLE_AGENT_CONFIG_CANDIDATE,
3235         )
3236     )
3237     return exp_id, ["gpt-4o-mini", "gpt-4o"]

```

```

3241 def main() -> None:
3242     import argparse
3243     import asyncio
3244     import sys
3245     parser = argparse.ArgumentParser(description="Seed AgentFlow-Eval demo data")
3246     parser.add_argument(
3247         "--force",
3248         action="store_true",
3249         help="Insert demo task even if other tasks already exist",
3250     )
3251     args = parser.parse_args()
3252     try:
3253         from app.core.dependencies import async_session_factory
3254     except Exception as exc:
3255         print(
3256             "Run from backend/ with venv activated and .env configured.",
3257             file=sys.stderr,
3258         )
3259         sys.exit(1)
3260     asyncio.run(seed_database(async_session_factory, force=args.force))
3261 if __name__ == "__main__":
3262     main()
3263 # === File: app/core/db/__init__.py ===
3264 """Database helpers: bulk queries and performance utilities."""
3265 from app.core.db.queries import (
3266     batch_suite_counts,
3267     count_suites_for_task,
3268     tasks_with_suite_counts,
3269 )
3270 __all__ = [
3271     "batch_suite_counts",
3272     "count_suites_for_task",
3273     "tasks_with_suite_counts",
3274 ]
3275 # === File: app/core/db/queries.py ===
3276 """Optimized query helpers – avoid N+1 patterns on hot list endpoints."""
3277 from __future__ import annotations
3278 from collections.abc import Sequence
3279 from typing import Any
3280 from sqlalchemy import Select, func, select
3281 from sqlalchemy.ext.asyncio import AsyncSession
3282 from app.models.task import Task
3283 from app.models.test_suite import TestSuite
3284 async def batch_suite_counts(
3285     session: AsyncSession,
3286     task_ids: Sequence[str],
3287 ) -> dict[str, int]:
3288     """Return ``{task_id: suite_count}`` in a single grouped query.
3289
3290     Args:
3291         session: Async DB session.
3292         task_ids: Task primary keys (empty → empty dict).
3293
3294     Returns:
3295         Mapping of task_id to number of test suites. Missing ids map to 0
3296         when looked up via ``.get(id, 0)``.
3297     """
3298     if not task_ids:
3299         return {}
3300     unique_ids = list(dict.fromkeys(task_ids))
3301     result = await session.execute(
3302         select(TestSuite.task_id, func.count(TestSuite.id))
3303         .where(TestSuite.task_id.in_(unique_ids))
3304         .group_by(TestSuite.task_id)
3305     )
3306     return {str(row[0]): int(row[1]) for row in result.all()}

```

```

3313 async def count_suites_for_task(session: AsyncSession, task_id: str) -> int:
3314     """Count suites for one task (detail views)."""
3315     result = await session.execute(
3316         select(func.count(TestSuite.id)).where(TestSuite.task_id == task_id)
3317     )
3318     return int(result.scalar() or 0)
3319 async def tasks_with_suite_counts(
3320     session: AsyncSession,
3321     tasks: Sequence[Task],
3322 ) -> list[tuple[Task, int]]:
3323     """Attach suite counts to a page of tasks without N+1 queries.

3325     Args:
3326         session: Async DB session.
3327         tasks: Already-fetched Task rows (e.g. one list page).

3329     Returns:
3330         List of `(task, suite_count)` in the same order as `tasks`.
3331     """
3332     counts = await batch_suite_counts(session, [t.id for t in tasks])
3333     return [(t, counts.get(t.id, 0)) for t in tasks]
3334 def apply_task_list_filters(
3335     query: Select[Any],
3336     *,
3337     status: str | None = None,
3338     include_archived: bool = False,
3339 ) -> Select[Any]:
3340     """Apply common list filters used by `GET /tasks`.
3341     if not include_archived:
3342         query = query.where(Task.is_archived.is_(False))
3343     if status:
3344         query = query.where(Task.status == status)
3345     return query
3346 # === File: app/models/__init__.py ===
3347 """ Task TestSuite Trace MetricScore ORM """
3348 from app.models.base import Base
3349 from app.models.task import Task
3350 from app.models.test_suite import TestSuite
3351 from app.models.trace import Trace
3352 from app.models.metric_score import MetricScore
3353 from app.models.audit_log import AuditLog
3354 from app.models.experiment import Experiment, ExperimentRun
3355 from app.models.media_asset import MediaAsset
3356 from app.models.ab_test import ABAssignment, ABEvent, ABExperiment, ABVariant
3357 from app.models.billing import (
3358     BillingPlan,
3359     Invoice,
3360     QuotaBalance,
3361     Subscription,
3362     UsageRecord,
3363 )
3364 from app.models.slow_task import SlowTaskEvent
3365 from app.models.agent_log import AgentLog
3366 from app.models.tenant import Tenant, TenantMember
3367 from app.models.benchmark import (
3368     Benchmark,
3369     BenchmarkCase,
3370     BenchmarkResult,
3371     BenchmarkRun,
3372 )

```

```

3385 __all__ = [
3386     "Base",
3387     "Task",
3388     "TestSuite",
3389     "Trace",
3390     "MetricScore",
3391     "AuditLog",
3392     "Experiment",
3393     "ExperimentRun",
3394     "MediaAsset",
3395     "ABExperiment",
3396     "ABVariant",
3397     "ABAssignment",
3398     "ABEvent",
3399     "BillingPlan",
3400     "Subscription",
3401     "UsageRecord",
3402     "QuotaBalance",
3403     "Invoice",
3404     "SlowTaskEvent",
3405     "AgentLog",
3406     "Tenant",
3407     "TenantMember",
3408     "Benchmark",
3409     "BenchmarkCase",
3410     "BenchmarkRun",
3411     "BenchmarkResult",
3412 ]
3413 # === File: app/models/agent_log.py ===
3414 """Agent observability log events (AOLS Phase 4 durable sink)."""
3415 from sqlalchemy import Index, JSON, String
3416 from sqlalchemy.orm import Mapped, mapped_column
3417 from app.models.base import Base, PKMixin, TimestampMixin
3418 class AgentLog(PKMixin, TimestampMixin, Base):
3419     """Structured runtime log row for query / statistics APIs.
3420
3421     ``payload`` holds the full event envelope (tokens, latency, agent_context, ...).
3422     """
3423     __tablename__ = "agent_logs"
3424     __table_args__ = (
3425         Index("ix_agent_logs_created_at", "created_at"),
3426         Index("ix_agent_logs_event_created", "event", "created_at"),
3427         Index("ix_agent_logs_level_created", "level", "created_at"),
3428         Index("ix_agent_logs_task_created", "task_id", "created_at"),
3429         Index("ix_agent_logs_trace_created", "trace_id", "created_at"),
3430         Index("ix_agent_logs_service", "service"),
3431     )
3432     level: Mapped[str] = mapped_column(
3433         String(16),
3434         nullable=False,
3435         default="info",
3436         comment="info|warning|error|debug",
3437     )
3438     event: Mapped[str] = mapped_column(
3439         String(96),

```



```

3457         nullable=False,
3458         index=True,
3459         comment="dot-notation event name",
3460     )
3461     service: Mapped[str] = mapped_column(
3462         String(64),
3463         nullable=False,
3464         default="agentflow-api",
3465         comment="emitting service",
3466     )
3467     trace_id: Mapped[str | None] = mapped_column(
3468         String(64),
3469         nullable=True,
3470         default=None,
3471         comment="correlation / request id",
3472     )
3473     task_id: Mapped[str | None] = mapped_column(
3474         String(64),
3475         nullable=True,
3476         default=None,
3477         comment="evaluation task id if any",
3478     )
3479     payload: Mapped[dict] = mapped_column(
3480         JSON,
3481         nullable=False,
3482         default=dict,
3483         comment="full event fields (redacted)",
3484     )
3485     def __repr__(self) -> str:
3486         return f"<AgentLog {self.event} level={self.level}>"
3487 # === File: app/models/audit_log.py ===
3488 """Audit log model for security-relevant actions."""
3489 from sqlalchemy import Index, JSON, String, Text
3490 from sqlalchemy.orm import Mapped, mapped_column
3491 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
3492 class AuditLog(PKMixin, TenantMixin, TimestampMixin, Base):
3493     """Immutable-style audit record (no update API)."""
3494     __tablename__ = "audit_logs"
3495     __table_args__ = (
3496         Index("ix_audit_logs_created_at", "created_at"),
3497         Index("ix_audit_logs_action", "action"),
3498         Index("ix_audit_logs_resource", "resource_type", "resource_id"),
3499         Index("ix_audit_logs_actor_created", "actor", "created_at"),
3500         Index("ix_audit_logs_tenant_created", "tenant_id", "created_at"),
3501     )
3502     actor: Mapped[str] = mapped_column(
3503         String(100),
3504         nullable=False,
3505         default="anonymous",
3506         comment="",
3507     )
3508     action: Mapped[str] = mapped_column(
3509         String(64),
3510         nullable=False,
3511         comment="", task.create / task.execute",

```

```

3529     )
3530     resource_type: Mapped[str] = mapped_column(
3531         String(64),
3532         nullable=False,
3533         comment="",
3534     )
3535     resource_id: Mapped[str | None] = mapped_column(
3536         String(64),
3537         nullable=True,
3538         default=None,
3539         comment="ID",
3540     )
3541     detail: Mapped[dict | None] = mapped_column(
3542         JSON,
3543         nullable=True,
3544         default=None,
3545         comment="",
3546     )
3547     request_id: Mapped[str | None] = mapped_column(
3548         String(64),
3549         nullable=True,
3550         default=None,
3551         comment="ID",
3552     )
3553     ip_address: Mapped[str | None] = mapped_column(
3554         String(64),
3555         nullable=True,
3556         default=None,
3557         comment="IP",
3558     )
3559     note: Mapped[str] = mapped_column(
3560         Text,
3561         nullable=False,
3562         default="",
3563         comment="",
3564     )
3565     def __repr__(self) -> str:
3566         return f"<AuditLog {self.action} {self.resource_type}/{self.resource_id}>"
3567 # === File: app/models/base.py ===
3568 """ ORM UUID """
3569 from datetime import datetime
3570 from uuid import uuid4
3571 from sqlalchemy import DateTime, String, func
3572 from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
3573 class Base(DeclarativeBase):
3574     """ ORM """
3575     pass
3576 class TimestampMixin:
3577     """ created_at / updated_at """
3578
3579     """
3580     created_at: Mapped[datetime] = mapped_column(
3581         DateTime(timezone=True),
3582         server_default=func.now(),
3583         comment="",
3584     )
3585     updated_at: Mapped[datetime | None] = mapped_column(
3586         DateTime(timezone=True),
3587         onupdate=func.now(),
3588         nullable=True,
3589         comment="",
3590     )
3591     )

```

```

3601 class PKMixin:
3602     """
3603         SQLite      PostgreSQL
3604     """
3605     id: Mapped[str] = mapped_column(
3606         String(36),
3607         primary_key=True,
3608         default=lambda: str(uuid4()),
3609         comment="UUID ",
3610     )
3611
3612 class TenantMixin:
3613     """Optional enterprise tenant scope (nullable for Lite / pre-migration rows)."""
3614     tenant_id: Mapped[str | None] = mapped_column(
3615         String(36),
3616         nullable=True,
3617         index=True,
3618         default=None,
3619         comment=" ID( ); = / ",
3620     )
3621 # === File: app/models/benchmark.py ===
3622 """Industry benchmark suite models."""
3623 from __future__ import annotations
3624 from datetime import datetime
3625 from sqlalchemy import DateTime, Float, ForeignKey, Index, Integer, JSON, String, Text
3626 from sqlalchemy.orm import Mapped, mapped_column, relationship
3627 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
3628 class Benchmark(PKMixin, TenantMixin, TimestampMixin, Base):
3629     """Named benchmark suite (collection of cases)."""
3630     __tablename__ = "benchmarks"
3631     __table_args__ = (
3632         Index("ix_benchmarks_created_by", "created_by"),
3633         Index("ix_benchmarks_status", "status"),
3634         Index("ix_benchmarks_tenant_created", "tenant_id", "created_at"),
3635     )
3636     name: Mapped[str] = mapped_column(String(255), nullable=False)
3637     description: Mapped[str] = mapped_column(Text, nullable=False, default="")
3638     status: Mapped[str] = mapped_column(
3639         String(32), nullable=False, default="active"
3640     )
3641     created_by: Mapped[str] = mapped_column(
3642         String(100), nullable=False, default="anonymous"
3643     )
3644     tags: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
3645     meta: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3646     cases: Mapped[list[BenchmarkCase]] = relationship(
3647         back_populates="benchmark",
3648         cascade="all, delete-orphan",
3649         order_by="BenchmarkCase.created_at",
3650     )
3651     runs: Mapped[list[BenchmarkRun]] = relationship(
3652         back_populates="benchmark",
3653         cascade="all, delete-orphan",
3654         order_by="BenchmarkRun.created_at",
3655     )

```

```

3673 class BenchmarkCase(PKMixin, TenantMixin, TimestampMixin, Base):
3674     """Single evaluation case inside a benchmark."""
3675     __tablename__ = "benchmark_cases"
3676     __table_args__ = (
3677         Index("ix_benchmark_cases_benchmark", "benchmark_id"),
3678         Index("ix_benchmark_cases_tenant", "tenant_id"),
3679     )
3680     benchmark_id: Mapped[str] = mapped_column(
3681         String(36),
3682         ForeignKey("benchmarks.id", ondelete="CASCADE"),
3683         nullable=False,
3684     )
3685     name: Mapped[str] = mapped_column(String(255), nullable=False, default="")
3686     user_query: Mapped[str] = mapped_column(Text, nullable=False)
3687     expected_output: Mapped[str] = mapped_column(Text, nullable=False, default="")
3688     expected_tools: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
3689     weight: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
3690     extra_metadata: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3691     benchmark: Mapped[Benchmark] = relationship(back_populates="cases")
3692 class BenchmarkRun(PKMixin, TenantMixin, TimestampMixin, Base):
3693     """One execution of a benchmark against an agent configuration."""
3694     __tablename__ = "benchmark_runs"
3695     __table_args__ = (
3696         Index("ix_benchmark_runs_benchmark", "benchmark_id"),
3697         Index("ix_benchmark_runs_task", "task_id"),
3698         Index("ix_benchmark_runs_status", "status"),
3699         Index("ix_benchmark_runs_tenant", "tenant_id"),
3700     )
3701     benchmark_id: Mapped[str] = mapped_column(
3702         String(36),
3703         ForeignKey("benchmarks.id", ondelete="CASCADE"),
3704         nullable=False,
3705     )
3706     task_id: Mapped[str | None] = mapped_column(String(36), nullable=True, index=True)
3707     label: Mapped[str] = mapped_column(
3708         String(128), nullable=False, default="default"
3709     )
3710     agent_config: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3711     status: Mapped[str] = mapped_column(
3712         String(32), nullable=False, default="pending"
3713     )
3714     created_by: Mapped[str] = mapped_column(
3715         String(100), nullable=False, default="anonymous"
3716     )
3717     started_at: Mapped[datetime | None] = mapped_column(
3718         DateTime(timezone=True), nullable=True
3719     )
3720     finished_at: Mapped[datetime | None] = mapped_column(
3721         DateTime(timezone=True), nullable=True
3722     )
3723     summary: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3724     benchmark: Mapped[Benchmark] = relationship(back_populates="runs")
3725     results: Mapped[list[BenchmarkResult]] = relationship(
3726         back_populates="run",
3727         cascade="all, delete-orphan",
3728     )

```

```

3745 class BenchmarkResult(PKMixin, TenantMixin, TimestampMixin, Base):
3746     """Per-case metrics for a benchmark run."""
3747     __tablename__ = "benchmark_results"
3748     __table_args__ = (
3749         Index("ix_benchmark_results_run", "run_id"),
3750         Index("ix_benchmark_results_case", "case_id"),
3751         Index("ix_benchmark_results_tenant", "tenant_id"),
3752     )
3753     run_id: Mapped[str] = mapped_column(
3754         String(36),
3755         ForeignKey("benchmark_runs.id", ondelete="CASCADE"),
3756         nullable=False,
3757     )
3758     case_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
3759     trace_id: Mapped[str | None] = mapped_column(String(36), nullable=True)
3760     accuracy: Mapped[float | None] = mapped_column(Float, nullable=True)
3761     quality: Mapped[float | None] = mapped_column(Float, nullable=True)
3762     latency_ms: Mapped[float | None] = mapped_column(Float, nullable=True)
3763     cost: Mapped[float | None] = mapped_column(Float, nullable=True)
3764     tokens: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
3765     score: Mapped[float | None] = mapped_column(Float, nullable=True)
3766     detail: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3767     run: Mapped[BenchmarkRun] = relationship(back_populates="results")
3768 # === File: app/models/billing.py ===
3769 """SaaS billing models: plans, subscriptions, usage, quotas, invoices."""
3770 from __future__ import annotations
3771 from datetime import datetime
3772 from sqlalchemy import (
3773     Boolean,
3774     DateTime,
3775     Float,
3776     Index,
3777     Integer,
3778     JSON,
3779     String,
3780     Text,
3781     UniqueConstraint,
3782     func,
3783 )
3784 from sqlalchemy.orm import Mapped, mapped_column
3785 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
3786 class BillingPlan(PKMixin, TimestampMixin, Base):
3787     """Subscription plan catalog."""
3788     __tablename__ = "billing_plans"
3789     __table_args__ = (UniqueConstraint("code", name="uq_billing_plans_code"),)
3790     code: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
3791     name: Mapped[str] = mapped_column(String(128), nullable=False)
3792     description: Mapped[str] = mapped_column(Text, nullable=False, default="")
3793     price_month_cents: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
3794     billing_cycle: Mapped[str] = mapped_column(
3795         String(32), nullable=False, default="monthly"
3796     )
3797     token_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=100_000)
3798     task_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
3799     storage_quota_mb: Mapped[int] = mapped_column(Integer, nullable=False, default=1024)
3800     plugin_quota: Mapped[int] = mapped_column(Integer, nullable=False, default=10)
3801     features: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3802     is_public: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
3803     is_active: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)

```

```

3817 class Subscription(PKMixin, TenantMixin, TimestampMixin, Base):
3818     """Actor/tenant subscription to a plan."""
3819     __tablename__ = "subscriptions"
3820     __table_args__ = (
3821         Index("ix_subscriptions_actor_status", "actor", "status"),
3822         Index("ix_subscriptions_tenant_status", "tenant_id", "status"),
3823     )
3824     actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
3825     plan_id: Mapped[str] = mapped_column(String(36), nullable=False, index=True)
3826     status: Mapped[str] = mapped_column(
3827         String(32), nullable=False, default="active"
3828     )
3829     period_start: Mapped[datetime | None] = mapped_column(
3830         DateTime(timezone=True), nullable=True
3831     )
3832     period_end: Mapped[datetime | None] = mapped_column(
3833         DateTime(timezone=True), nullable=True
3834     )
3835     external_ref: Mapped[str | None] = mapped_column(String(128), nullable=True)
3836 class UsageRecord(PKMixin, TenantMixin, Base):
3837     """Immutable usage event (token / task / judge / storage)."""
3838     __tablename__ = "usage_records"
3839     __table_args__ = (
3840         Index("ix_usage_actor_created", "actor", "created_at"),
3841         Index("ix_usage_metric_created", "metric", "created_at"),
3842         Index("ix_usage_trace_id", "trace_id"),
3843         Index("ix_usage_tenant_created", "tenant_id", "created_at"),
3844     )
3845     actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
3846     metric: Mapped[str] = mapped_column(
3847         String(32), nullable=False
3848     )
3849     quantity: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
3850     unit_cost: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
3851     ref_type: Mapped[str | None] = mapped_column(String(64), nullable=True)
3852     ref_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
3853     trace_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
3854     extra: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
3855     created_at: Mapped[datetime] = mapped_column(
3856         DateTime(timezone=True),
3857         server_default=func.now(),
3858         nullable=False,
3859     )
3860 class QuotaBalance(PKMixin, TimestampMixin, Base):
3861     """Period-scoped counters per actor."""
3862     __tablename__ = "quota_balances"
3863     __table_args__ = (
3864         UniqueConstraint("actor", "period", name="uq_quota_actor_period"),
3865         Index("ix_quota_actor_period", "actor", "period"),
3866     )
3867     actor: Mapped[str] = mapped_column(String(100), nullable=False)
3868     period: Mapped[str] = mapped_column(String(7), nullable=False)
3869     token_used: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
3870     token_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=100_000)
3871     task_used: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
3872     task_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
3873     storage_used_mb: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
3874     storage_limit_mb: Mapped[int] = mapped_column(Integer, nullable=False, default=1024)
3875     plugin_used: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
3876     plugin_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=10)

```

```

3889 class Invoice(PKMixin, TenantMixin, TimestampMixin, Base):
3890     """Billing period invoice (draft until payment integration)."""
3891     __tablename__ = "invoices"
3892     __table_args__ = (
3893         Index("ix_invoices_actor_period", "actor", "period"),
3894         Index("ix_invoices_tenant_period", "tenant_id", "period"),
3895     )
3896     actor: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
3897     period: Mapped[str] = mapped_column(String(7), nullable=False)
3898     amount_cents: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
3899     status: Mapped[str] = mapped_column(
3900         String(32), nullable=False, default="draft"
3901     )
3902     line_items: Mapped[list] = mapped_column(JSON, nullable=False, default=list)
3903     issued_at: Mapped[datetime | None] = mapped_column(
3904         DateTime(timezone=True), nullable=True
3905     )
3906 # === File: app/models/experiment.py ===
3907 """Experiment models – multi-run comparison on the same test suites."""
3908 from __future__ import annotations
3909 from typing import TYPE_CHECKING, Any
3910 from sqlalchemy import ForeignKey, Index, JSON, String, Text, UniqueConstraint
3911 from sqlalchemy.orm import Mapped, mapped_column, relationship
3912 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
3913 if TYPE_CHECKING:
3914     from app.models.task import Task
3915 class Experiment(PKMixin, TenantMixin, TimestampMixin, Base):
3916     """A comparison experiment grouping multiple evaluation runs.
3917
3918     Suites are snapshotted as JSON so later base-task edits do not alter
3919     historical comparisons. Each run materializes a Task with cloned suites.
3920     """
3921     __tablename__ = "experiments"
3922     __table_args__ = (
3923         Index("ix_experiments_created_by", "created_by"),
3924         Index("ix_experiments_created_at", "created_at"),
3925         Index("ix_experiments_base_task_id", "base_task_id"),
3926         Index("ix_experiments_owner_created", "created_by", "created_at"),
3927         Index("ix_experiments_tenant_created", "tenant_id", "created_at"),
3928     )
3929     name: Mapped[str] = mapped_column(String(255), nullable=False, comment=" ")
3930     description: Mapped[str] = mapped_column(
3931         Text, nullable=False, default="", comment=" "
3932     )
3933     base_task_id: Mapped[str | None] = mapped_column(
3934         String(36),
3935         nullable=True,
3936         comment=" : ID",
3937     )
3938     suite_snapshot: Mapped[list[dict[str, Any]]] = mapped_column(
3939         JSON,
3940         nullable=False,
3941         default=list,
3942         comment=" [{user_query, expected_output, expected_tools}]",
3943     )
3944     created_by: Mapped[str] = mapped_column(
3945         String(100),
3946         nullable=False,
3947         default="anonymous",
3948         comment=" actor",
3949     )
3950     runs: Mapped[list[ExperimentRun]] = relationship(
3951         back_populates="experiment",
3952         cascade="all, delete-orphan",
3953         order_by="ExperimentRun.created_at",
3954     )

```

```

3961     def __repr__(self) -> str:
3962         return f"<ExperimentRun id={self.id} name={self.name!r}>"
3963 class ExperimentRun(PKMixin, TenantMixin, TimestampMixin, Base):
3964     """One evaluation run inside an experiment (maps 1:1 to a Task)."""
3965     __tablename__ = "experiment_runs"
3966     __table_args__ = (
3967         UniqueConstraint("experiment_id", "label", name="uq_experiment_run_label"),
3968         Index("ix_experiment_runs_tenant", "tenant_id"),
3969     )
3970     experiment_id: Mapped[str] = mapped_column(
3971         String(36),
3972         ForeignKey("experiments.id", ondelete="CASCADE"),
3973         nullable=False,
3974         index=True,
3975         comment=" ",
3976     )
3977     task_id: Mapped[str] = mapped_column(
3978         String(36),
3979         ForeignKey("tasks.id", ondelete="CASCADE"),
3980         nullable=False,
3981         index=True,
3982         comment=" ",
3983     )
3984     label: Mapped[str] = mapped_column(
3985         String(100),
3986         nullable=False,
3987         comment=" ", gpt-4o / http-agent-v1",
3988     )
3989     agent_config: Mapped[dict[str, Any]] = mapped_column(
3990         JSON,
3991         nullable=False,
3992         default=dict,
3993         comment=" run agent_config ",
3994     )
3995     experiment: Mapped[Experiment] = relationship(back_populates="runs")
3996     task: Mapped[Task] = relationship()
3997     def __repr__(self) -> str:
3998         return f"<ExperimentRun id={self.id} label={self.label!r} task={self.task_id}>"
3999 # === File: app/models/media_asset.py ===
4000 """MediaAsset – uploaded multimodal files linked to tasks/suites."""
4001 from __future__ import annotations
4002 from typing import TYPE_CHECKING, Any
4003 from sqlalchemy import ForeignKey, Index, Integer, JSON, String, Text
4004 from sqlalchemy.orm import Mapped, mapped_column, relationship
4005 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
4006 if TYPE_CHECKING:
4007     from app.models.task import Task
4008     from app.models.test_suite import TestSuite
4009 class MediaAsset(PKMixin, TenantMixin, TimestampMixin, Base):
4010     """Stored multimodal artifact with extraction results."""
4011     __tablename__ = "media_assets"
4012     __table_args__ = (
4013         Index("ix_media_assets_task_id", "task_id"),
4014         Index("ix_media_assets_suite_id", "test_suite_id"),
4015         Index("ix_media_assets_created_by", "created_by"),

```



```

4033         Index("ix_media_assets_kind", "media_kind"),
4034         Index("ix_media_assets_tenant", "tenant_id"),
4035     )
4036     filename: Mapped[str] = mapped_column(String(512), nullable=False)
4037     content_type: Mapped[str] = mapped_column(
4038         String(128), nullable=False, default="application/octet-stream"
4039     )
4040     size_bytes: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
4041     sha256: Mapped[str] = mapped_column(String(64), nullable=False, default="")
4042     media_kind: Mapped[str] = mapped_column(String(32), nullable=False, default="other")
4043     storage_backend: Mapped[str] = mapped_column(
4044         String(32), nullable=False, default="local"
4045     )
4046     storage_key: Mapped[str] = mapped_column(String(1024), nullable=False)
4047     extracted_text: Mapped[str] = mapped_column(Text, nullable=False, default="")
4048     features: Mapped[dict[str, Any] | None] = mapped_column(
4049         JSON, nullable=True, default=None
4050     )
4051     extract_meta: Mapped[dict[str, Any] | None] = mapped_column(
4052         JSON, nullable=True, default=None
4053     )
4054     task_id: Mapped[str | None] = mapped_column(
4055         String(36),
4056         ForeignKey("tasks.id", ondelete="SET NULL"),
4057         nullable=True,
4058     )
4059     test_suite_id: Mapped[str | None] = mapped_column(
4060         String(36),
4061         ForeignKey("test_suites.id", ondelete="SET NULL"),
4062         nullable=True,
4063     )
4064     created_by: Mapped[str] = mapped_column(
4065         String(100), nullable=False, default="anonymous"
4066     )
4067     task: Mapped[Task | None] = relationship()
4068     test_suite: Mapped[TestSuite | None] = relationship()
4069     def __repr__(self) -> str:
4070         return (
4071             f"<MediaAsset id={self.id} kind={self.media_kind!r} file={self.filename!r}>"
4072         )
4073 # === File: app/models/metric_score.py ===
4074 """MetricScore model """
4075 from typing import TYPE_CHECKING
4076 from sqlalchemy import Boolean, Float, ForeignKey, Index, JSON, String, Text
4077 from sqlalchemy.orm import Mapped, mapped_column, relationship
4078 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
4079 if TYPE_CHECKING:
4080     from app.models.trace import Trace
4081 class MetricScore(PKMixin, TenantMixin, TimestampMixin, Base):
4082     """
4083
4084         Trace
4085         LLM
4086     """
4087     __tablename__ = "metric_scores"

```

```

4105 __table_args__ = (
4106     Index("ix_metric_scores_trace_metric", "trace_id", "metric_name"),
4107     Index("ix_metric_scores_human_reviewed", "is_human_reviewed"),
4108     Index("ix_metric_scores_tenant", "tenant_id"),
4109 )
4110 trace_id: Mapped[str] = mapped_column(
4111     ForeignKey("traces.id", ondelete="CASCADE"),
4112     nullable=False,
4113     index=True,
4114     comment="FK to trace",
4115 )
4116 metric_name: Mapped[str] = mapped_column(
4117     String(100),
4118     nullable=False,
4119     comment="metric name",
4120 )
4121 score: Mapped[float] = mapped_column(
4122     Float,
4123     nullable=False,
4124     comment="score 0-100",
4125 )
4126 reason: Mapped[str] = mapped_column(
4127     Text,
4128     nullable=False,
4129     default="",
4130     comment="deduction reason",
4131 )
4132 extra_metadata: Mapped[dict | None] = mapped_column(
4133     JSON,
4134     nullable=True,
4135     default=None,
4136     comment="extra metadata",
4137 )
4138 confidence: Mapped[float | None] = mapped_column(
4139     Float,
4140     nullable=True,
4141     default=None,
4142     comment="LLM      0.0 ~ 1.0",
4143 )
4144 is_human_reviewed: Mapped[bool] = mapped_column(
4145     Boolean,
4146     nullable=False,
4147     default=False,
4148     comment="",
4149 )
4150 human_score: Mapped[float | None] = mapped_column(
4151     Float,
4152     nullable=True,
4153     default=None,
4154     comment="(      LLM      )",
4155 )
4156 reviewer: Mapped[str | None] = mapped_column(
4157     String(100),
4158     nullable=True,
4159     default=None,
4160     comment="",
4161 )
4162 trace: Mapped["Trace"] = relationship(back_populates="metric_scores")
4163 @property

```

```

4177     def effective_score(self) -> float:
4178         """
4179         if self.is_human_reviewed and self.human_score is not None:
4180             return self.human_score
4181         return self.score
4182     def __repr__(self) -> str:
4183         return f"<MetricScore name={self.metric_name!r} score={self.score} reviewed={self.
4184 # === File: app/models/slow_task.py ===
4185 """Persisted slow-task samples for diagnostics across restarts."""
4186 from __future__ import annotations
4187 from datetime import datetime
4188 from sqlalchemy import DateTime, Float, Index, JSON, String, func
4189 from sqlalchemy.orm import Mapped, mapped_column
4190 from app.models.base import Base, PKMixin
4191 class SlowTaskEvent(PKMixin, Base):
4192     """One slow agent/judge/pipeline sample above configured threshold."""
4193     __tablename__ = "slow_task_events"
4194     __table_args__ = (
4195         Index("ix_slow_task_created", "created_at"),
4196         Index("ix_slow_task_stage_created", "stage", "created_at"),
4197         Index("ix_slow_task_trace", "trace_id"),
4198         Index("ix_slow_task_actor", "actor"),
4199     )
4200     stage: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
4201     duration_sec: Mapped[float] = mapped_column(Float, nullable=False)
4202     threshold_sec: Mapped[float] = mapped_column(Float, nullable=False)
4203     ref_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
4204     status: Mapped[str] = mapped_column(String(32), nullable=False, default="ok")
4205     trace_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
4206     actor: Mapped[str | None] = mapped_column(String(100), nullable=True)
4207     extra: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
4208     created_at: Mapped[datetime] = mapped_column(
4209         DateTime(timezone=True),
4210         server_default=func.now(),
4211         nullable=False,
4212     )
4213 # === File: app/models/task.py ===
4214 """Task
4215 import enum
4216 from typing import TYPE_CHECKING
4217 from sqlalchemy import Boolean, Index, JSON, Enum, String, Text
4218 from sqlalchemy.orm import Mapped, mapped_column, relationship
4219 from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin
4220 if TYPE_CHECKING:
4221     from app.models.test_suite import TestSuite
4222 class TaskStatus(str, enum.Enum):
4223     """
4224
4225     :
4226     CREATED → QUEUED → RUNNING → JUDGING → COMPLETED
4227
4228         ↓           ↓
4229         FAILED     FAILED
4230
4231         ↓
4232         CANCELLED / TIMEOUT
4233
4234     """
4235     CREATED = "created"
4236     QUEUED = "queued"
4237     RUNNING = "running"
4238     WAITING_TOOL = "waiting_tool"
4239     JUDGING = "judging"
4240     COMPLETED = "completed"
4241     FAILED = "failed"
4242     CANCELLED = "cancelled"
4243     TIMEOUT = "timeout"
4244     @classmethod

```

```

4249     def allowed_transitions(cls) -> dict["TaskStatus", set["TaskStatus"]]:
4250         """
4251         return {
4252             cls.CREATED: {cls.QUEUED, cls.CANCELLED},
4253             cls.QUEUED: {cls.RUNNING, cls.CANCELLED, cls.TIMEOUT},
4254             cls.RUNNING: {
4255                 cls.WAITING_TOOL,
4256                 cls.JUDGING,
4257                 cls.FAILED,
4258                 cls.CANCELLED,
4259                 cls.TIMEOUT,
4260             },
4261             cls.WAITING_TOOL: {cls.RUNNING, cls.FAILED, cls.CANCELLED, cls.TIMEOUT},
4262             cls.JUDGING: {cls.COMPLETED, cls.FAILED, cls.CANCELLED},
4263             cls.COMPLETED: set(),
4264             cls.FAILED: set(),
4265             cls.CANCELLED: set(),
4266             cls.TIMEOUT: set(),
4267         }
4268     def can_transition_to(self, target: "TaskStatus") -> bool:
4269         """
4270         return target in self.allowed_transitions().get(self, set())
4271 class Task(PKMixin, TenantMixin, TimestampMixin, Base):
4272     """
4273
4274     Task      TestSuite
4275     """
4276     __tablename__ = "tasks"
4277     __table_args__ = (
4278         Index(
4279             "ix_tasks_owner_archived_created",
4280             "created_by",
4281             "is_archived",
4282             "created_at",
4283         ),
4284         Index("ix_tasks_status_created", "status", "created_at"),
4285         Index("ix_tasks_created_by", "created_by"),
4286         Index("ix_tasks_celery_task_id", "celery_task_id"),
4287         Index("ix_tasks_is_archived", "is_archived"),
4288         Index("ix_tasks_tenant_created", "tenant_id", "created_at"),
4289     )
4290     name: Mapped[str] = mapped_column(
4291         String(255),
4292         nullable=False,
4293         comment=" ",
4294     )
4295     description: Mapped[str] = mapped_column(
4296         Text,
4297         nullable=False,
4298         default="",
4299         comment=" ",
4300     )
4301     status: Mapped[TaskStatus] = mapped_column(
4302         Enum(
4303             TaskStatus,

```